

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY



BACHELOR THESIS

By
Nguyễn Gia Huy - 23BI14194

Title:
AI-Powered CV Screening System (RecruitIQ)

External Supervisor: Hoàng Trọng Hiếu
Internal Supervisor: Giang Anh Tuấn

Hanoi, July 2026

To whom it may concern,

I, Hoàng Trọng Hiếu, certify that the thesis/ internship report of Mr/Ms. Nguyễn Gia Huy is qualified to be presented in the Internship July 2025-2026.

Hanoi, July, 2026

Supervisor's signature

Table of Contents

Table of Contents.....	3
ACKNOWLEDGEMENTS.....	5
LIST OF ABBREVIATIONS.....	6
LIST OF TABLES.....	7
LIST OF FIGURES.....	8
ABSTRACT.....	9
I/ INTRODUCTION.....	10
1.1. Product Background.....	10
1.2. Existing Solutions.....	10
1.2.1 Manual Spreadsheet + Email Workflow.....	10
1.2.2 Enterprise ATS Platforms (e.g., Workable, Greenhouse, Lever).....	11
1.2.3 LinkedIn Recruiter / Job Board Sourcing Tools.....	12
1.3. Proposed Solution.....	13
1.4. Thesis Structure.....	13
II/ OBJECTIVES.....	15
2.1. Major Features.....	15
2.2 Limitations & Exclusions.....	16
2.3. Expected Outcome.....	16
2.3.1 Business & Operational Impact.....	16
2.3.2 User Experience Impact.....	16
2.3.3 Planned Next Steps After Project Completion.....	17
III/ Requirement Analysis.....	18
3.1 System Overview.....	18
3.1.1 Context Diagram.....	18
3.1.2 Main Business Flows.....	19
3.2 Actors and User Roles.....	21
3.3 User Requirements.....	22
3.3.1 Use Case Diagrams.....	22
3.3.2 Use Case Specification.....	23
3.4 Functional Requirements.....	40
3.4.1 System Features.....	40
3.4.2 Screen Inventory.....	42
3.5 Non-Functional Requirements.....	43
3.6 Conceptual Data Model.....	45

3.6.1 Entity Relationship Diagram (ERD).....	45
3.6.2 Entity Description.....	45
3.7 AI Scoring Algorithm.....	47
IV/ MATERIALS AND METHODS.....	47
4.1 Technology Stack.....	47
4.2 System Architecture.....	49
4.2.1 Client Tier.....	50
4.2.2 Application Tier.....	51
4.2.3 Data Tier.....	54
4.3 AI Processing Pipeline Design.....	54
4.4 Database Design.....	56
4.4.1 Database Schema.....	56
4.4.2 Database Schemas Description.....	56
4.4.3 Data Dictionary.....	57
4.5 Security Design.....	64
4.6 REST API Design.....	64
V/ RESULTS AND DISCUSSION.....	66
5.1 Results.....	66
5.1.1 Implemented System Overview.....	66
5.1.2 Unit and Integration Test Results.....	66
5.1.3 AI Pipeline Evaluation.....	66
5.1.4 Non-Functional Requirements Evaluation.....	67
5.2 Discussion.....	67
VI/ CONCLUSION & PERSPECTIVE.....	68
6.1 Summary of Contributions.....	68
6.2 Limitations.....	69
6.3 Future Work.....	70
6.4 Closing Remarks.....	70
REFERENCES.....	71

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to all those who gave me the possibility to complete this thesis, both through the specific support provided during this project and the broader academic foundation I received throughout my studies at the University of Science and Technology of Hanoi (USTH).

Foremost, I would like to extend my honest thanks to my internal supervisor, Giang Anh Tuấn, for their continuous guidance, constructive feedback, and patience throughout the development of this thesis. Their advice on both the technical architecture and the academic framing of this work was invaluable.

I am equally grateful to my external supervisor, Hoàng Trọng Hiếu of FPT software, for the opportunity to develop this project in a real or simulated industry context, and for the mentorship provided during the internship period.

I would also like to thank the faculty members of the Department of Information and Communication Technology at USTH for the rigorous training in software engineering, system design, and artificial intelligence that formed the foundation for this work, as well as my classmates and colleagues who provided feedback during testing.

Finally, I wish to thank my family for their unconditional support throughout my years of study.

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
CV	Curriculum Vitae
DOCX	Office Open XML Document Format
DTO	Data Transfer Object
HR	Human Resources
ICT	Information and Communication Technology
JD	Job Description
JPA	Java Persistence API
JWT	JSON Web Token
LLM	Large Language Model
NFR	Non-Functional Requirement
NLP	Natural Language Processing
ORM	Object-Relational Mapping
OTP	One-Time Password
PDF	Portable Document Format
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
USTH	University of Science and Technology of Hanoi

LIST OF TABLES

Table 1: Profile of the Manual Candidate Tracking Approach	11
Table 2: Analysis of Enterprise ATS Platforms	12
Table 3: Analysis of Job Board Sourcing and Recruitment Tools	12
Table 4: Overview of RecruitIQ Major Features	15
Table 5: System Limitations and Future Exclusions	16
Table 6: Operational Sequence of the HR Officer Workflow	20
Table 7: Candidate Application Workflow and Operational Steps	21
Table 8: System Actors	21
Table 9: View Public Job List	24
Table 10: View Public Job Detail	24
Table 11: Register User Account	25
Table 12: Sign In	26
Table 13: Reset Password	27
Table 14: View/Update User Profile	28
Table 15: Sign Out	29
Table 16: Apply Job	29
Table 17: Track My Applications	30
Table 18: Rate Candidate's CV(Candidate)	31
Table 19: View HR Dashboard	32
Table 20: View Job List	32
Table 21: Add New Job	33
Table 22: Update Job Detail	34
Table 23: View Job Applications	34
Table 24: Add Candidate's CV	35
Table 25: Rate Candidate's CV (HR rating)	36
Table 26: Track Candidate's CV (view rating history)	36
Table 27: View Admin Dashboard	37
Table 28: View Account List	37
Table 29: Add New Account	38
Table 30: View/Update Account Detail	39
Table 31: Activate/Deactivate Account	40
Table 32: Functional Requirements by Module	41
Table 33: Screen Description	43
Table 34: Non Functional Requirements	44
Table 35: User Description	45
Table 36: Job Post Description	46
Table 37: Candidate Description	46
Table 38: CV Detail Description	46
Table 39: Score Record Description	46
Table 40: AI Scoring Weight Distribution	47

Table 41: Technology Stack	48
Table 42: System architecture	50
Table 43: User Data	58
Table 44: Password Reset Tokens Data	58
Table 45: City Data	59
Table 46: Job Data	60
Table 47: Candidate Data	60
Table 48: Parsed_profiles Data	61
Table 49: Score_records Data	62
Table 50: Ai_summaries Data	63
Table 51: Shortlists Data	63
Table 52: Key REST API Endpoints	65
Table 53: Test Results	66
Table 54: Non-Functional Requirements Evaluation Results	67

LIST OF FIGURES

Figure 1: Context Diagram	18
Figure 2:HR Business Flow Diagram	19
Figure 3:Apply CV Business Flow Diagram	20
Figure 4:Guest Use Case Diagram	22
Figure 5:Sign-In User Use Case Diagram	22
Figure 6:Candidate Use Case Diagram	22
Figure 7:HR Officer Use Case Diagram	23
Figure 8:Admin Use Case Diagram	23
Figure 9: Screen Flow	42
Figure 10: Entity Relationship Diagram	45
Figure 11: System Architecture	49
Figure 12: Database Schema	56

ABSTRACT

Recruitment screening is one of the most time-intensive activities in human resource management. A single job posting at a technology company may attract fifty to two hundred applications within a week, requiring HR officers to spend three to five hours manually reading and evaluating CVs for each open position. This process is slow and inherently inconsistent - different reviewers apply different standards, and the absence of an objective scoring method means equally qualified candidates can receive different outcomes depending on who reviews their CV and when.

This thesis presents RecruitIQ, an AI-powered CV screening and candidate ranking platform built to automate and standardise the initial screening phase of recruitment. The system integrates a large language model (Google Gemini) into an asynchronous processing pipeline that extracts structured information from uploaded CV files (PDF and DOCX), scores each candidate against a job posting's specific requirements using a transparent weighted algorithm - skills (35%), experience (30%), education (15%), certifications (10%), and soft skills (10%) - and produces a concise AI-generated assessment with a recommendation label (STRONG_MATCH, POTENTIAL_MATCH, or NOT_RECOMMENDED).

The system was implemented as a full-stack web application using Spring Boot 3.2 (Java 17) for the backend REST API and React 19 (TypeScript) for the frontend, with PostgreSQL as the primary data store and Cloudinary for CV file storage. Three roles are supported - HR Officer, Candidate, and Admin - each with a dedicated interface.

Evaluation against a test set of CVs showed approximately 80% agreement between AI recommendations and expert HR judgement, with average end-to-end processing time of 17.3 seconds per CV, comfortably within the defined performance targets.

Key words: AI-powered recruitment, CV screening, candidate ranking, natural language processing, large language model, Spring Boot, React, applicant tracking.

I/ INTRODUCTION

1.1. Product Background

HR officers at technology companies and growing organisations receive dozens - sometimes hundreds - of CVs for each open position. A single job posting can attract fifty to two hundred applications within a week of going live. Currently, most HR teams rely on a manual review process: each CV is opened, skimmed, and mentally compared against the job requirements. A typical HR officer spends three to five hours screening fifty CVs for a single role - time that cannot be spent on interviews, candidate engagement, or strategic hiring work. When multiple positions are open simultaneously, the backlog becomes unmanageable.

Manual CV screening is also fundamentally inconsistent. Different HR officers apply different standards for the same role. Even the same officer evaluates CVs differently depending on fatigue, order effects, and subjective impressions. There are no standardised scoring criteria, no audit trail, and no way to compare candidates objectively across a batch. Qualified candidates with non-traditional backgrounds or formatting styles are frequently overlooked, while candidates who format their CVs to match keywords advance despite weaker actual qualifications.

Beyond accuracy, the current approach does not scale. As hiring volume grows - particularly in technology roles where demand for skilled engineers, data scientists, and product managers is high - manual screening becomes the bottleneck that slows every subsequent stage of recruitment. Interview slots go unfilled, good candidates accept offers elsewhere while waiting for a response, and the organisation's employer brand suffers from slow feedback loops. A company actively hiring for ten or more positions simultaneously can lose weeks of productivity per position to this bottleneck alone.

The problem is not a lack of effort - it is the absence of a systematic, AI-assisted tool that can ingest CVs, extract structured information, score candidates objectively against job requirements, and present HR officers with a ranked shortlist in minutes rather than days. This is the gap that RecruitIQ is designed to fill.

1.2. Existing Solutions

1.2.1 Manual Spreadsheet + Email Workflow

Many HR teams, particularly at small and mid-sized companies, manage candidate pipelines using a combination of email (to receive CVs) and shared spreadsheets (to track candidates). HR officers manually download attachments, open each CV, record notes in a spreadsheet row, and assign a subjective rating.

Attribute	Description
Website / Link	N/A - ad hoc combination of tools

Target Users	Small HR teams with limited budget
Core Features	Free, flexible, no setup required; familiar to all users
Strengths	Zero cost; no vendor dependency; accessible from any device
Weaknesses	No scoring algorithm; no structured extraction; inconsistent ratings; cannot handle high volume; no audit trail; CV files scattered across email inboxes; no role-based access control
GAP for this project	The spreadsheet approach produces no comparable, objective scores. When fifty CVs arrive, the HR officer must still read every one. There is no mechanism to rank candidates by their fit for specific job requirements, and no AI assistance to extract key data. As volume grows, the approach breaks down entirely - which is the exact scaling threshold RecruitIQ targets.

Table 1: Profile of the Manual Candidate Tracking Approach

1.2.2 Enterprise ATS Platforms (e.g., Workable, Greenhouse, Lever)

Enterprise Applicant Tracking Systems such as Workable, Greenhouse, and Lever are full-featured platforms used by large HR teams globally. They manage the entire recruitment lifecycle - from job posting through offer letter - with integrations to job boards, calendaring tools, and HRIS systems.

Attribute	Description
Website / Link	https://workable.com / https://greenhouse.io / https://lever.co
Target Users	Mid-to-large enterprises with dedicated HR teams and multi-step hiring pipelines
Core Features	Applicant tracking, job board publishing, interview scheduling, collaborative review, offer management, analytics
Strengths	Comprehensive workflow coverage; strong integrations; mobile apps; compliance features
Weaknesses	High subscription cost (typically USD 200–600/month at entry tier); complex setup and onboarding; CV screening is keyword-based, not AI-weighted scoring; no automatic candidate ranking by fit score; designed for large teams with multiple collaborators, not optimised for single-HR-officer workflows
GAP for this project	Enterprise ATS platforms solve workflow management, not CV quality assessment. They do not automatically score a

	candidate's skills, experience level, and education against the job's specific requirements and produce a weighted fit score. An HR officer using Workable still reads every CV - the ATS just organises where those CVs are stored. RecruitIQ's core value proposition - AI-generated scoring and ranking - is not available in any standard ATS tier.
--	---

Table 2: Analysis of Enterprise ATS Platforms

1.2.3 LinkedIn Recruiter / Job Board Sourcing Tools

LinkedIn Recruiter and similar sourcing tools (VietnamWorks Premium, TopCV, Indeed Resume) allow HR officers to search for candidates and view their profiles. Some offer basic candidate matching based on keyword overlap with a job description.

Attribute	Description
Website / Link	https://linkedin.com/talent / https://vietnamworks.com / https://topcv.vn
Target Users	HR teams focused on active sourcing and outreach; companies publishing job postings externally
Core Features	Candidate search, InMail outreach, job posting distribution, basic profile matching
Strengths	Access to large candidate databases; good for proactive sourcing; some AI-powered matching on premium tiers
Weaknesses	Designed for sourcing - not for screening CVs that candidates submit directly; no ability to upload and score a batch of PDF/DOCX files; matching is keyword-based and opaque, not a transparent weighted score; expensive for premium features; no integration with internal HR workflows
GAP for this project	These tools work at the top of the funnel (finding candidates) but provide no systematic help once CVs have arrived. When an HR officer receives fifty email attachments from a job posting, LinkedIn Recruiter is irrelevant - there is nothing to search. RecruitIQ addresses the post-application screening phase that sourcing tools ignore entirely.

Table 3: Analysis of Job Board Sourcing and Recruitment Tools

Section Summary - Unmet Need

Manual workflows are free but do not scale and produce inconsistent, subjective results. Enterprise ATS platforms organise CVs but do not score them - an HR officer still reads every application manually. Sourcing tools address a different phase of recruitment entirely and cannot process inbound CV batches. None of

these solutions automatically extracts structured data from uploaded CVs, scores each candidate against specific job requirements using a transparent weighted algorithm, and presents a ranked shortlist with AI-generated reasoning. That is the specific gap RecruitIQ fills.

1.3. Proposed Solution

RecruitIQ is an AI-powered CV screening platform that automates the most time-consuming phase of recruitment: the initial screening of inbound applications. When an HR officer uploads a batch of CVs for a job posting, the system immediately begins an asynchronous processing pipeline - extracting structured information from each CV using a large language model, scoring each candidate against the job's specific requirements (skills, experience, education, certifications, and soft skills) using a transparent weighted algorithm, and generating a concise AI summary with a recommendation (STRONG_MATCH, POTENTIAL_MATCH, or NOT_RECOMMENDED). The HR officer does not wait - results appear progressively as each CV is processed.

The solution directly closes the gaps identified in Section 1.2. It gives HR officers a ranked candidate list with objective, comparable scores rather than a pile of unread PDF files. It eliminates the three to five hours of manual reading per role and replaces it with a structured review of AI-generated summaries, score breakdowns, and reasoning. HR officers review candidates in order of fit score, investigate the AI's reasoning where needed, and make final decisions (shortlist, hold, or reject) with full context - all from a single interface. Candidates who self-apply through the public portal receive the same AI processing automatically, so self-submitted applications and HR-uploaded CVs are evaluated on identical criteria.

RecruitIQ is purpose-built for the reality that most HR teams in Japan and similar markets face: a single HR officer managing multiple open positions, moderate-to-high application volumes, and no budget for enterprise ATS subscriptions. The system is designed to be lightweight, role-specific, and immediately operational - an HR officer can create a job posting, upload CVs, and see AI-generated rankings within minutes of first use, without training or configuration. Candidates benefit from a clean public portal and transparent application tracking. Admins manage user access without complexity.

1.4. Thesis Structure

This thesis is organized into six core chapters, transitioning from the initial problem definition and requirement analysis to the system design, implementation results, and subsequent evaluation. The detailed content of each chapter is outlined below:

- **Chapter I: Introduction**

This chapter establishes the foundational context of the product by detailing the current challenges faced by HR departments in high-volume recruitment. It reviews existing market alternatives-such as manual spreadsheet workflows,

enterprise Applicant Tracking Systems (ATS), and external sourcing tools-highlighting their respective operational gaps. Finally, it introduces RecruitIQ as the proposed AI-powered solution to automate and standardize the initial CV screening process.

- **Chapter II: Objectives**

This chapter defines the explicit scope of the project. It enumerates the major functional features of the platform, details the technical and operational boundaries (limitations and exclusions), and delineates the expected business, operational, and user experience impacts upon deployment. It concludes with the strategic next steps planned for subsequent iterations.

- **Chapter III: Requirement Analysis**

This chapter provides a comprehensive breakdown of the system's functional and technical prerequisites. It presents the system overview via a Context Diagram and maps out the two primary end-to-end business flows: HR-Led AI Screening and Candidate Self-Application. Furthermore, it specifies system actors, detailed Use Case specifications (including main, alternative, and exception paths), screen inventories, background job behaviors, non-functional constraints, and the conceptual data model (Entity-Relationship Diagram) governing the system.

- **Chapter IV: Materials and Methods**

This chapter details the technical implementation blueprint and architectural choices. It clarifies the selected technology stack (Java, Spring Boot, React, TypeScript, PostgreSQL, and Google Gemini API) along with the core rationale for each choice. It describes the decoupled four-tier system architecture, the event-driven asynchronous design of the AI processing pipeline, the fully normalized relational database schema (including an exhaustive data dictionary), and the stateless JWT security and access control mechanisms.

- **Chapter V: Results and Discussion**

This chapter delivers an empirical evaluation of the fully implemented RecruitIQ application. It provides quantitative results from unit and integration testing across core service classes and protected API endpoints. It further presents the metrics from the AI pipeline evaluation-assessing extraction accuracy, processing latency, and consistency against human expert judgment-and discusses the technical trade-offs, constraints, and error-handling resilience observed during testing.

- **Chapter VI: Conclusion & Perspective**

The final chapter synthesizes the primary technical and academic contributions of the thesis. It explicitly addresses the current limitations of the platform, such as language processing biases and external LLM dependencies, and proposes concrete directions for future research and development, including advanced pipeline analytics, configurable scoring weights, and localized multi-language models.

II/ OBJECTIVES

2.1. Major Features

ID	Feature Name	Description
1	AI-Powered CV Screening Pipeline	After each CV upload (by HR or self-apply), the system automatically parses the CV text, scores the candidate against job requirements using a weighted algorithm (skills 35%, experience 30%, education 15%, certifications 10%, soft skills 10%), and generates a summary with a recommendation.
2	Ranked Candidate List with Filters	HR officers see all candidates for a job ranked by AI score. The list is filterable by minimum score, minimum experience, and education level. Recommendation badges (STRONG_MATCH / POTENTIAL_MATCH / NOT_RECOMMENDED) are visible at a glance.
3	Candidate Detail with Score Breakdown	HR officers can expand any candidate to see the full parsed profile (extracted skills, experience, education), the score breakdown per dimension with AI reasoning (strengths, gaps), and the complete AI summary.
4	HR Decision Management	HR officers mark each candidate as SHORTLISTED, ON_HOLD, or REJECTED, optionally adding internal notes. Decisions are recorded with the deciding officer's identity and timestamp.
5	Job Posting Management	HR officers create structured job postings with required skills, minimum experience years, required education level, and a job description text. These fields are used directly by the AI scoring engine.
6	Candidate Self-Apply Portal	Candidates register, browse OPEN job listings publicly (no sign-in required), self-apply by uploading their own CV, and track their application status and AI processing result.
7	Export & Download	HR officers can download individual CV files.
8	Admin User Management	Admins create, edit, and activate/deactivate HR_Officer and Candidate accounts. Global statistics are available to Admins.

Table 4: Overview of RecruitIQ Major Features

2.2 Limitations & Exclusions

ID	Excluded Item	Reason
1	Interview scheduling	Interview coordination (calendar invites, room booking, interviewer assignment) is a separate workflow that requires integration with calendaring services.
2	Integration with external job boards	Publishing job postings to LinkedIn, VietnamWorks, or TopCV is out of scope.
3	Multi-language support	The system is in English only.
4	Native mobile application	A responsive web application covers mobile browser use cases.
5	Advanced analytics and reporting charts	The system provides basic counts and exports.
6	AI job description generator	Generating JD text from a role title using AI is a useful feature but not required for the core screening workflow.

Table 5: System Limitations and Future Exclusions

2.3. Expected Outcome

2.3.1 Business & Operational Impact

By replacing manual CV reading with an AI-powered screening pipeline, RecruitIQ is expected to reduce initial screening time by approximately 80% - from three to five hours per role to thirty to sixty minutes of reviewing AI-generated summaries and making decisions. For an organisation with five to ten active job postings, this recovery represents fifteen to forty hours of HR officer time per week that can be redirected toward candidate engagement, structured interviewing, and employer branding. The transparent, weighted scoring algorithm eliminates the category of inconsistency where two CVs of equivalent quality receive different outcomes based on the order they were read or which officer reviewed them. With all candidate data centralized and exportable, the organisation gains a complete audit trail of every screening decision - who reviewed whom, when, and with what outcome.

2.3.2 User Experience Impact

HR officers gain a single dashboard that shows every candidate for every open position ranked by AI fit score, replacing the current experience of navigating email folders and updating spreadsheet rows. Each candidate card displays the recommendation badge, score, and key extracted data at a glance - and clicking through reveals the full AI reasoning, allowing the HR officer to agree, probe further, or override with a note. Candidates gain a clean, modern self-apply portal where they can see exactly which stage

their CV is in (processing, completed, or error) and what the AI's recommendation was - replacing the anxiety of submitting a CV and hearing nothing for days. Admins gain a user management panel that removes the need for manual account provisioning via database or backend scripts.

2.3.3 Planned Next Steps After Project Completion

Following successful system deployment and an initial pilot period, the planned next steps include:

- Interview scheduling module - allow HR officers to assign interviewers, book slots, and track interview outcomes within the same platform;
- Job board integrations - publish job postings directly to VietnamWorks, TopCV, and LinkedIn, and ingest applications automatically;
- Analytics dashboard - visualise time-to-hire, screening funnel conversion rates per stage, and AI score distribution across positions; and
- AI job description generator - allow HR officers to generate structured JD text from a role title and key requirements, reducing the time to post a new position.

III/ Requirement Analysis

3.1 System Overview

3.1.1 Context Diagram

RecruitIQ is an AI-powered CV screening and candidate ranking platform that automates the initial screening phase of recruitment. The system consists of two deployed components: a Spring Boot REST API backend and a React single-page application frontend. Three user roles interact with the system - HR Officers, Candidates, and Administrators - plus unauthenticated Guests who can browse public job listings without signing in.

The core value of the system is delivered through an asynchronous AI processing pipeline: every CV uploaded to the system (whether by an HR Officer uploading a batch or by a Candidate self-applying) is automatically processed by a three-phase pipeline that (1) extracts structured information from the CV text using a large language model, (2) scores the candidate against the job's specific requirements using a transparent weighted algorithm, and (3) generates a concise AI-authored assessment with a recommendation label. The pipeline runs in the background so that HR Officers can continue working while results are produced progressively.

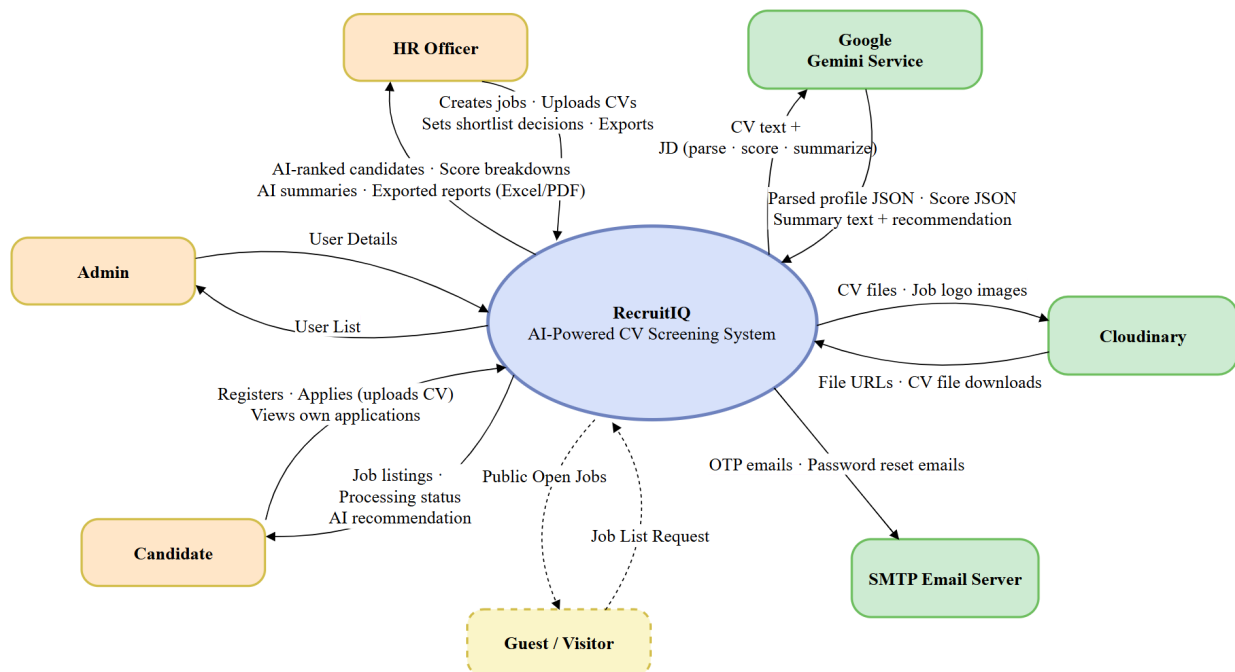


Figure 1: Context Diagram

3.1.2 Main Business Flows

The system supports three primary end-to-end business flows:

a. BF-01: HR-Led AI Screening

Trigger: HR_Officer decides to open a new position and creates a job posting.

End condition: All uploaded CVs have been processed by AI and the HR_Officer has made a shortlist decision on each candidate.

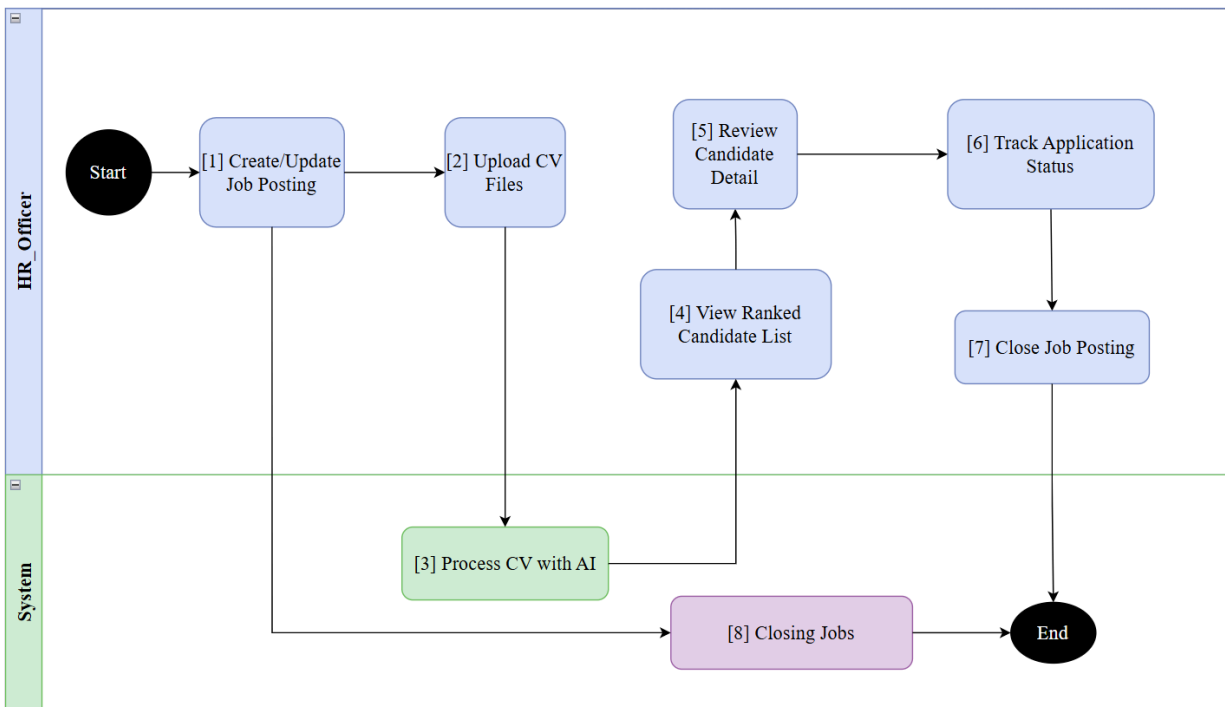


Figure 2:HR Business Flow Diagram

Step	Actor	Action
1	HR_Officer	Create/Update Job Posting: creates a new job posting or update an existing job posting with requirements (required skills, min experience, required education, job description text)
2	HR_Officer	Upload CV Files: uploads one CV file (PDF/DOCX) for the job
3	System	Process CV with AI: parses structured data, scores against job requirements, generates AI summary and recommendation
4	HR_Officer	View Ranked Candidate List: views the ranked candidate list filtered by score and recommendation
5	HR_Officer	Reviews Candidate Detail: parsed profile, score breakdown, AI reasoning

Step	Actor	Action
6	HR_Officer	Tracking Application Status: Marks each candidate SHORTLISTED, ON_HOLD, or REJECTED with optional notes
7	HR_Officer	Close Job Posting: closes the job posting when hiring is complete
8	System	Closing Jobs: background job to auto-close jobs when deadline passes

Table 6: Operational Sequence of the HR Officer Workflow

Exception paths:

- Step 3: AI processing fails (network error, LLM API error) → candidate status set to ERROR; HR_Officer sees error indicator in candidate list; other candidates continue processing normally.
- Step 6: HR_Officer changes a decision after initial marking => the system overwrites the previous decision and records the new timestamp.
- Step 8: Job deadline passes before HR_Officer closes manually => system auto-closes the job on schedule.

b. BF-02: Candidate Self-Application

Trigger: A Candidate registers an account on the platform.

End condition: Candidate has seen the AI recommendation for all submitted applications and has no pending actions.

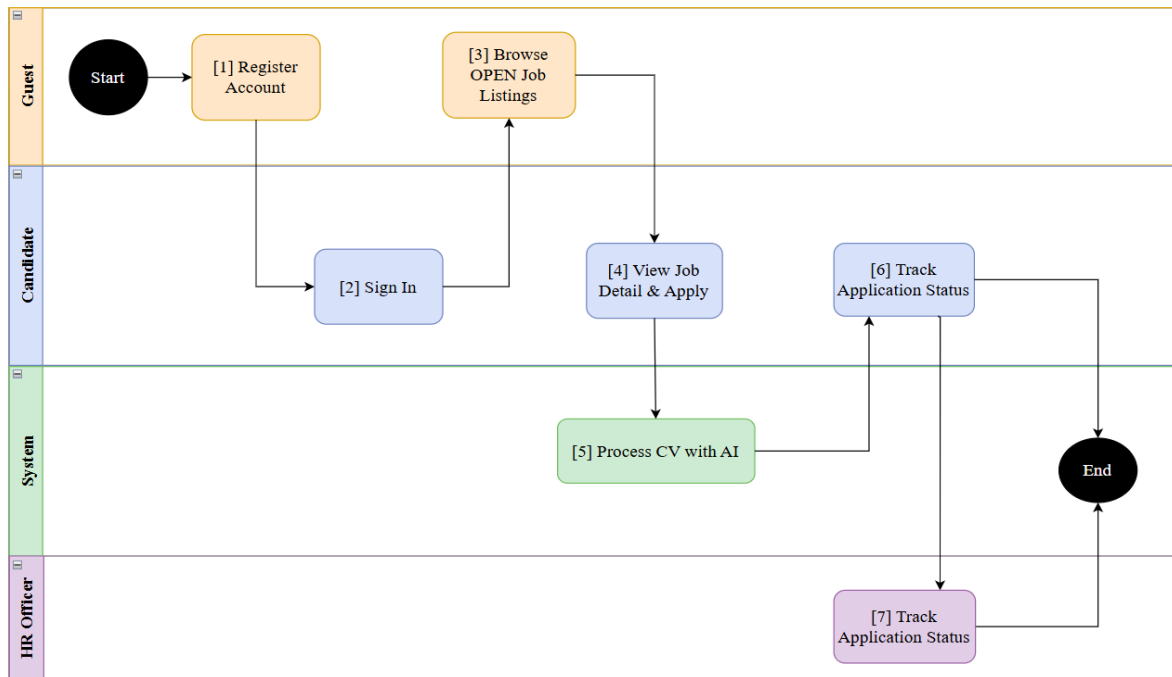


Figure 3: Apply CV Business Flow Diagram

Step	Actor	Action
1	Candidate	Register Account: registers an account with name, email, and password
2	Candidate	Signs In: logs in to the system with email and password.
3	Candidate	Browse OPEN Job Listings: view available job posts
4	Candidate	View Job Detail & Apply: views job detail and submits application with CV upload
5	System	Process CV with AI: parses structured data, scores against job requirements, generates AI summary and recommendation
6	HR_Officer	Tracks application status: marks each candidate SHORTLISTED, ON_HOLD, or REJECTED with optional notes; the system then send email notification to the candidate
7	Candidate	Tracks Application Status: view application status, withdraw application

Table 7: Candidate Application Workflow and Operational Steps

3.2 Actors and User Roles

The system defines four actor types:

Actor	Description	Sign-In Required	Notes
Admin	Manages all user accounts and has access to global platform statistics. Inherits all HR Officer capabilities.	Yes	Created in seed data / by another Admin; not self-registerable
HR Officer	Creates job postings, uploads CV batches, reviews AI screening results, and makes shortlist decisions.	Yes	Created by Admin
Candidate	Self-registers, browses public job listings, submits applications by uploading their own CV, and tracks processing status.	Yes	Self-registers with OTP email verification
Guest	Browse public OPEN job listings and job detail pages without signing in. Read-only access.	No	Read-only access to job listings and job detail pages

Table 8: System Actors

3.3 User Requirements

3.3.1 Use Case Diagrams

a. Guest User

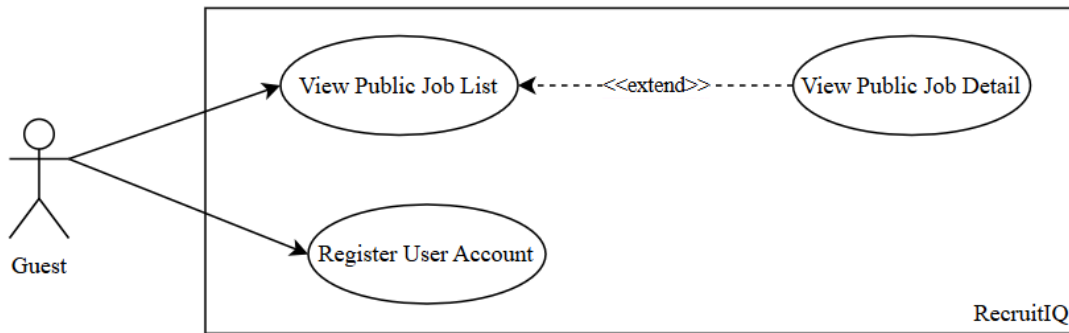


Figure 4: Guest Use Case Diagram

b. Sign-In User

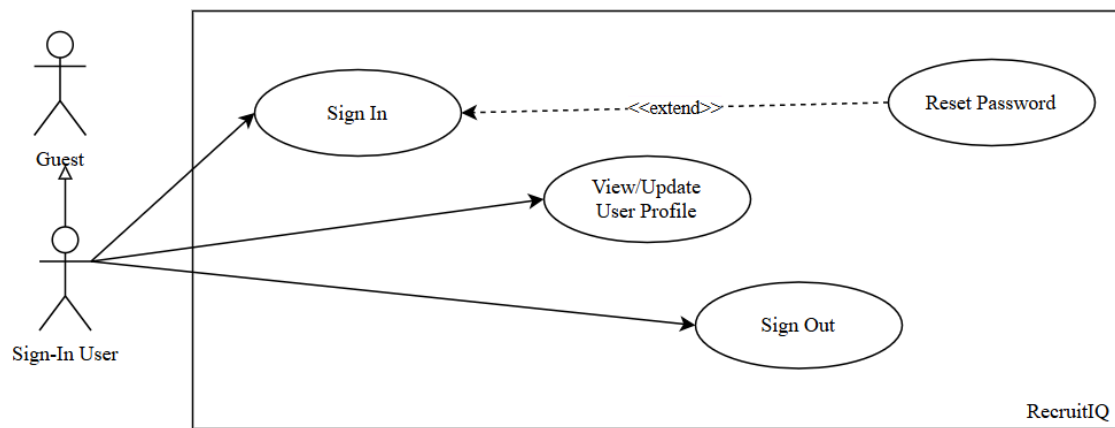


Figure 5: Sign-In User Use Case Diagram

c. Candidate User

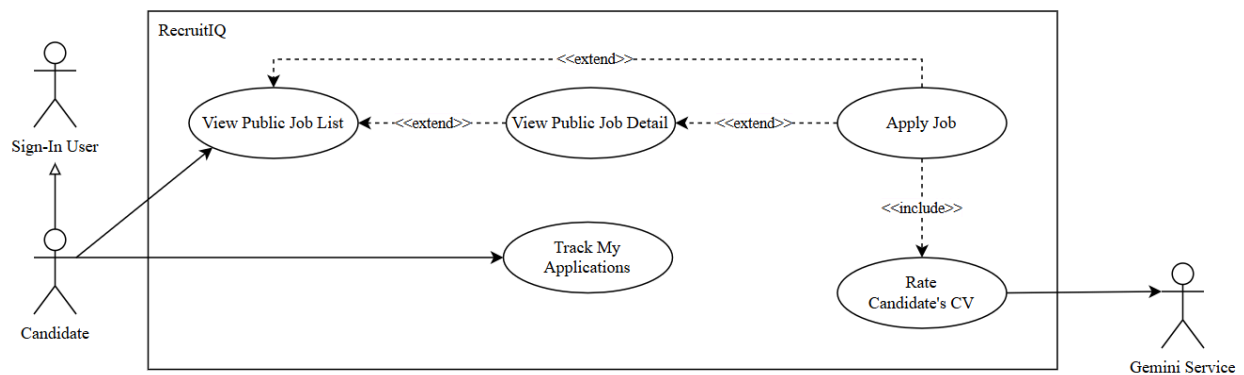


Figure 6: Candidate Use Case Diagram

d. HR Officer User

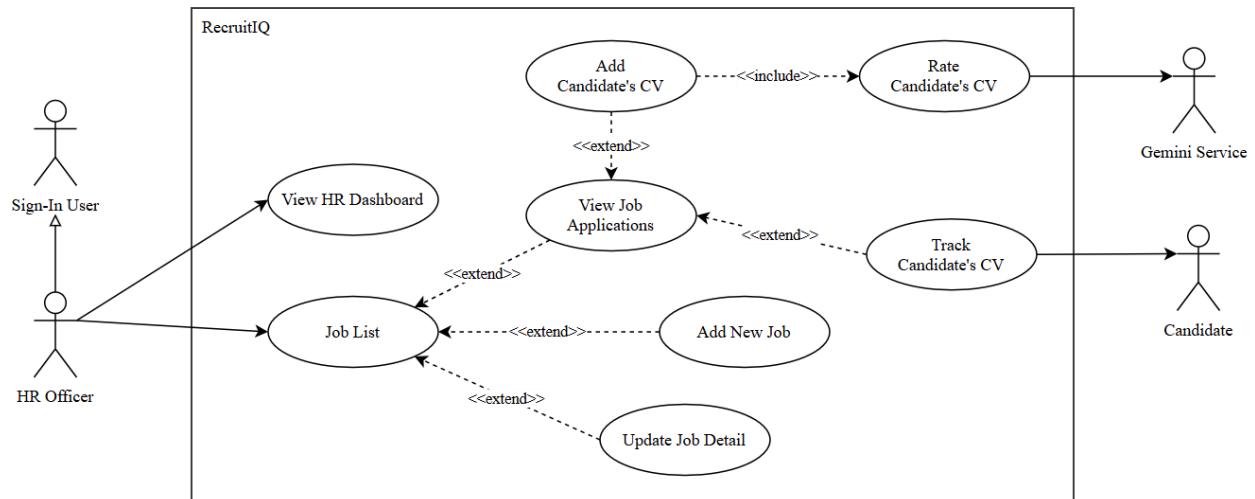


Figure 7:HR Officer Use Case Diagram

e. Admin User

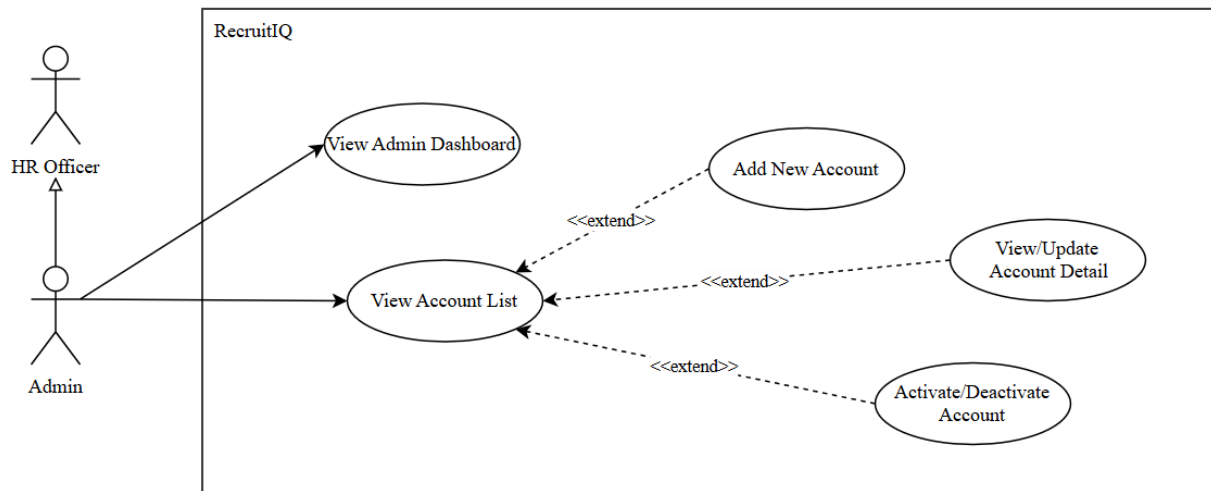


Figure 8:Admin Use Case Diagram

3.3.2 Use Case Specification

3.3.2.1 Guest

Field	Value
UC-01	View Public Job List
Primary Actor	Guest (no login)
Pre-conditions	At least one job exists with status PUBLISHED

Post-conditions	Guest sees a paginated list of jobs; no data is persisted
Allowed Roles	Public – no login required

Table 9: View Public Job List

Main Flow

1. [U] Guest opens the home page and selects Jobs from the navigation menu.
2. [S] System queries the database for all jobs where status = PUBLISHED.
3. [S] System returns the result set and renders a list view (title, company, location, short description).
4. [U] Guests may apply optional filters (location, category, date) and sort options.
5. [S] System re-queries with the filter criteria and updates the list view.

Alternative Flows

- AF-01 – No Jobs : When step 2 returns an empty set, the system displays “No public jobs are currently available.”
- AF-02 – Filter Error : If a filter value is invalid, the system shows an inline validation message and retains the previous list.

Business Rules

- BR-01 Only jobs with status = PUBLISHED are shown to the public.
- BR-02 Pagination defaults to 20 items per page; the guest can navigate pages.
- BR-03 Filtering fields must match the predefined enumeration (e.g., supported locations).

Field	Value
UC-02	View Public Job Detail
Primary Actor	Guest
Pre-conditions	Guests have a job identifier from the public list.
Post-conditions	The guest sees the full job description; no state change.
Allowed Roles	Public – no login required

Table 10: View Public Job Detail

Main Flow

1. [U] Guest clicks a job title on the list view.
2. [S] System retrieves the complete job record (id, title, description, requirements, benefits, company profile).
3. [S] System renders the Job Detail page.
4. [U] Guests may click Back to List to return.

Alternative Flows

- AF-01 – Job Not Found : If step 2 fails (job removed), system shows “Job not found or no longer available.”
- AF-02 – Unauthorized Access : If the job is mistakenly marked INTERNAL, the system returns a “403 – Access denied” page.

Business Rules

- BR-01: The detail page must include mandatory sections: Description, Requirements, Company Overview.
- BR-02: The page URL is SEO-friendly: /jobs/{slug}-{id}.

Field	Value
UC-03	Register User Account
Primary Actor	Guest (candidate)
Pre-conditions	No prior account exists for the supplied email address.
Post-conditions	A new CANDIDATE account is created with status INACTIVE and an OTP is sent to the e-mail address.
Allowed Roles	Public – no login required

Table 11: Register User Account

Main Flow

1. [U] The candidate navigates to the registration page and fills in their full name, email address, and password.
2. [S] Validates all fields - see BR-01, BR-02, BR-03.
3. [S] Checks that no active or inactive account already exists with the submitted email address - see BR-04.
4. [S] Creates a new user account with role CANDIDATE and status INACTIVE .
5. [S] Generates a 6-digit OTP, stores it with an expiry time, and sends it to the candidate’s email address.
6. [S] Displays a confirmation message instructing the candidate to check their email and enter the OTP to activate their account. The candidate is directed to the OTP verification page .

Alternative Flows

- AF-01 - Email already registered: At step 3, the system finds an existing account with the same email address. The candidate sees the message “An account with this email address already exists.” The form is displayed again with the email field highlighted.

- AF-02 - Validation errors: At step 2, one or more fields fail validation. The form is re-displayed with specific inline error messages for each failing field (e.g., “Password must be at least 8 characters and contain one uppercase letter and one digit”). No account is created.
- AF-03 - Email delivery failure: At step 5, the email service fails to deliver the OTP. The account is created with status ACTIVE. The candidate sees an error message “We could not send the verification email. Please use the Resend OTP option to try again.”

Business Rules

- BR-01: Full name is required and must be between 2 and 100 characters.
- BR-02: Email address is required and must be in valid email format.
- BR-03: Password is required, must be at least 8 characters long, contain at least one uppercase letter, and contain at least one digit.
- BR-04: Email addresses are unique across all accounts regardless of role or status. Duplicate registrations using the same email are not permitted.
- BR-05: The role assigned at self-registration is always CANDIDATE. HR_Officer and Admin accounts can only be created by an Admin (UC-21).

3.3.2.2 Sign-In User (Authenticated Candidate)

Field	Value
UC-04	Sign In
Primary Actor	Candidate
Pre-conditions	The user has a verified e-mail address and a password.
Post-conditions	An authenticated session is created; the user is redirected to Candidate Dashboard.
Allowed Roles	CANDIDATE, ADMIN, HR OFFICER

Table 12: Sign In

Main Flow

1. [U] Candidate enters e-mail and password on the login form and clicks Sign In.
2. [S] System validates the credentials (hash compare).
3. [S] If credentials are correct and the account status is ACTIVE, a session token (JWT or server-side) is generated.
4. [S] System stores the token in a secure HttpOnly cookie and redirects to the dashboard.

Alternative Flows

- AF-01 – Invalid Credentials The system returns “Incorrect e-mail or password.” and re-displays the login form.
- AF-02 – Inactive Account System shows “Account not activated. Please verify your e-mail.” with a Resend OTP link.
- AF-03 – Account Locked After 5 failed attempts, the system locks the account and displays a lock-out message.

Business Rules

- BR-01 Passwords are stored using bcrypt (or Argon2) with a work factor ≥ 12 .
- BR-02 Session tokens expire after 30 minutes of inactivity.
- BR-03 Brute-force protection: lock after 5 consecutive failures for 15 minutes.

Field	Value
UC-05	Reset Password
Primary Actor	Candidate
Pre-conditions	The candidate knows the registered e-mail address.
Post-conditions	Password is updated; candidates can sign in with the new password.
Allowed Roles	CANDIDATE, ADMIN, HR OFFICER

Table 13: Reset Password

Main Flow

1. [U] Candidate clicks Forgot password? and submits the e-mail address.
2. [S] System generates a one-time reset token (valid 15 min) and emails a reset link.
3. [U] Candidate opens the link, enters a new password (twice) and submits.
4. [S] System validates the token, updates the password hash, invalidates the token, and shows a success message.

Alternative Flows

- AF-01 – Invalid Email The system replies “No account found with that e-mail.”
- AF-02 – Expired Token The system shows “Reset link expired; request a new one.”
- AF-03 – Email Delivery Failure The system logs the failure and offers a Resend button.

Business Rules

- BR-01 New password must satisfy the same complexity rules as registration (≥ 8 chars, 1 uppercase, 1 digit).
- BR-02 Reset token is a cryptographically-random 32-byte string.

- BR-03 After a successful reset, all existing sessions for the user are revoked.

Field	Value
UC-06	View/Update User Profile
Primary Actor	Candidate
Pre-conditions	The candidate is authenticated (active session).
Post-conditions	Profile information is persisted; changes are reflected on subsequent reads.
Allowed Roles	CANDIDATE, ADMIN, HR OFFICER

Table 14: View/Update User Profile

Main Flow

1. [U] Candidate navigates to My Profile.
2. [S] System loads current profile data (name, phone, address, CV file).
3. [U] Candidate edits fields and/or uploads a new CV, then clicks Save.
4. [S] System validates each field (see BR-01-BR-04) and stores the updates.
5. [S] System displays a confirmation “Profile updated successfully.”

Alternative Flows

- AF-01 – Validation Error Inline error messages are shown; no data is saved.
- AF-02 – CV Upload Failure The system shows “File could not be uploaded; please try again.”

Business Rules

- BR-01 Name: 2-100 chars.
- BR-02 The phone number must match the pattern +{CountryCode} {Number}.
- BR-03 Uploaded CV must be PDF, max 5 MB.
- BR-04 Email cannot be modified by the candidate (only by admin).

Field	Value
UC-07	Sign Out
Primary Actor	Candidate
Pre-conditions	An active authenticated session exists.
Post-conditions	Session token is invalidated; user is redirected to the public home page.

Allowed Roles	CANDIDATE, ADMIN, HR OFFICER
---------------	------------------------------

Table 15: Sign Out

Main Flow

1. [U] Candidate clicks Sign Out (header or dropdown).
2. [S] System removes the session cookie / revokes the token.
3. [S] System redirects to the public landing page.

Alternative Flows – none (single step).

Business Rules

- BR-01 All “remember-me” tokens are also cleared on sign-out.

3.3.2.3 Candidate (Job-Seeker)

Field	Value
UC-08	Apply Job
Primary Actor	Candidate
Pre-conditions	The candidate is logged in and the target job status is OPEN.
Post-conditions	An Application record is created with status SUBMITTED; candidate receives a confirmation e-mail.
Allowed Roles	CANDIDATE

Table 16: Apply Job

Main Flow

1. [U] Candidate opens a job’s detail page and clicks Apply.
2. [S] System presents an application form (pre-filled with stored CV, optional cover-letter).
3. [U] Candidate reviews/edits the CV, adds a cover letter, and clicks Submit.
4. [S] System creates an Application entry, links it to the candidate and the job, sets status SUBMITTED, and stores a snapshot of the CV.
5. [S] System sends a confirmation e-mail with application ID and next-step info.
6. [S] System shows a “Thank you” page with a link to My Applications.

Alternative Flows

- AF-01 – Job Closed If in step 1 the job status is no longer OPEN, system shows “Applications for this job are closed.”
- AF-02 – Missing CV If the candidate has no CV uploaded, the system redirects to the Upload CV screen before allowing submission.

- AF-03 – Submission Error On DB failure, the system displays “We could not process your application. Please try again later.” and rolls back any partial data.

Business Rules

- BR-01 Only one ACTIVE application per candidate per job is allowed.
- BR-02 CV snapshot is stored as a PDF copy to preserve the state at submission time.
- BR-03 Application status transitions: SUBMITTED → UNDER_REVIEW → REJECTED / ACCEPTED.

Field	Value
UC-09	Track My Applications
Primary Actor	Candidate
Pre-conditions	The candidate is logged in and has at least one submitted application.
Post-conditions	The candidate sees a list of applications with current statuses; can view details of each.
Allowed Roles	CANDIDATE

Table 17: Track My Applications

Main Flow

1. [U] Candidate selects My Applications from the navigation menu.
2. [S] System retrieves all Application records for the candidate, ordered by submission date.
3. [S] System renders a table: Job Title, Submission Date, Current Status, Action (View Detail).
4. [U] Candidate clicks a row → [S] System displays the detailed view (CV snapshot, cover letter, HR comments, timeline).

Alternative Flows

- AF-01 – No Applications The system shows “You have not applied to any jobs yet.”
- AF-02 – Filter / Search Candidates may filter by status; system re-queries accordingly.

Business Rules

- BR-01 Only the candidate who submitted the application can view its details.
- BR-02 Status colors: Submitted (gray), Under Review (blue), Accepted (green), Rejected (red).

Field	Value
UC-10	Rate Candidate's CV (Candidate)
Primary Actor	Candidate
Pre-conditions	The candidate has an uploaded CV.
Post-conditions	A rating (1-5 stars) and optional comment are stored linked to the current CV version.
Allowed Roles	CANDIDATE

Table 18: Rate Candidate's CV(Candidate)

Main Flow

1. [U] Candidate opens My Profile → Rate My CV button.
2. [S] System shows a rating widget (stars) and a free-text comment box.
3. [U] Candidate selects a rating, optionally writes a comment, and clicks Submit.
4. [S] System validates (rating $\in$ 1..5), stores the rating tied to the CV version timestamp
5. [S] System displays “Your rating has been saved.”

Alternative Flows

- AF-01 – No CV System prompts “Upload a CV before rating.”
- AF-02 – Invalid Input The system forces the star rating (cannot be empty).

Business Rules

- BR-01 Ratings are immutable; a new rating creates a separate record (historical tracking).
- BR-02 Average rating per candidate is computed for recommendation algorithms.

3.3.2.4 HR Officer

Field	Value
UC-11	View HR Dashboard
Primary Actor	HR Officer
Pre-conditions	HR Officer is authenticated with the HR role.
Post-conditions	Dashboard shows key metrics (open jobs, pending applications, recent hires).

Allowed Roles	HR_OFFICER
---------------	------------

Table 19: View HR Dashboard

Main Flow

1. [U] HR Officer selects HR Dashboard from the sidebar.
2. [S] System aggregates data: count of OPEN jobs, count of PENDING applications, recent activity log.
3. [S] System renders charts (bar, line) and a quick-action panel (Create Job, Review Applications).

Alternative Flows – none (read-only).

Business Rules

- BR-01 Dashboard data refreshes every 5 minutes.
- BR-02 Only jobs belonging to the officer's department are shown (if multi-tenant).

Field	Value
UC-12	View Job List
Primary Actor	HR Officer
Pre-conditions	Authenticated HR Officer.
Post-conditions	List of jobs (Draft, Published, Closed) with edit links is displayed.
Allowed Roles	HR_OFFICER

Table 20: View Job List

Main Flow

1. [U] HR Officer navigates to the Jobs menu.
2. [S] System fetches all jobs the officer can manage, grouped by status.
3. [S] System renders a table with columns: Title, Status, Applicants, Actions (Edit, Close).

Alternative Flows

- AF-01 – No Jobs The system shows “You have not created any jobs yet.”
- AF-02 – Filter by Status The officer selects a filter; system re-queries and updates the list.

Business Rules

- BR-01 Only jobs with owner_id = current_user_id are editable.

- BR-02 Deleting a Published job is prohibited; it must be Closed first.

Field	Value
UC-13	Add New Job
Primary Actor	HR Officer
Pre-conditions	Authenticated HR Officer.
Post-conditions	A new job record exists with status DRAFT (or PUBLISHED if the officer chooses).
Allowed Roles	HR_OFFICER

Table 21: Add New Job

Main Flow

1. [U] HR Officer clicks Add New Job.
2. [S] System presents a job-creation wizard (Title, Department, Location, Description, Requirements, Salary, Employment type, Publishing date).
3. [U] Officer fills the form and selects Save as Draft or Publish.
4. [S] System validates mandatory fields (see BR-01-BR-04).
5. [S] System creates the job entry, sets the chosen status, and logs the creation event.
6. [S] System redirects to the Job List with a success toast “Job created successfully.”

Alternative Flows

- AF-01 – Validation Failure Inline error messages appear; no job is created.
- AF-02 – Publish Conflict If required fields for publishing are missing, the system forces Save as Draft and notifies the officer.

Business Rules

- BR-01 Title: 5-150 chars, unique per department.
- BR-02 Salary must be a positive number; optional range may be supplied.
- BR-03 Publishing date cannot be earlier than today.
- BR-04 Only HR Officers can set status PUBLISHED; otherwise default is DRAFT.
- BR-05 Job records are immutable after being CLOSED.

Field	Value
UC-14	Update Job Detail
Primary Actor	HR Officer

Pre-conditions	The job exists and belongs to the officer; status is DRAFT or PUBLISHED.
Post-conditions	The job record reflects the modifications; change audit is recorded.
Allowed Roles	HR_OFFICER

Table 22: Update Job Detail

Main Flow

1. [U] Officer selects Edit next to a job in the list.
2. [S] System loads current job data into the same wizard used for creation.
3. [U] Officer modifies fields and clicks Save.
4. [S] System validates changes (same rules as UC-13) and updates the DB.
5. [S] System logs the edit (who, when, what fields changed) and shows a success message.

Alternative Flows

- AF-01 – Attempt to Edit Closed Job The system shows “Closed jobs cannot be edited.”
- AF-02 – Validation Errors Same as UC-13, inline messages shown.

Business Rules – identical to UC-13 (BR-01 – BR-05).

Field	Value
UC-15	View Job Applications
Primary Actor	HR Officer
Pre-conditions	The job exists and the officer has permission to view its applications.
Post-conditions	The officer sees a list of candidate applications for that job, with status and CV preview links.
Allowed Roles	HR_OFFICER

Table 23: View Job Applications

Main Flow

1. [U] Officer opens a job’s detail page and clicks View Applications.
2. [S] System queries all Application records linked to the job.
3. [S] System displays a table: Candidate Name, Submission Date, Current Status, Actions (View CV, Rate CV, Change Status).

Alternative Flows

- AF-01 – No Applications The system shows “No candidates have applied yet.”
- AF-02 – Filter by Status The officer selects a status filter; system re-queries.

Business Rules

- BR-01 Only the HR Officer assigned to the job can view the applications.
- BR-02 Application data is read-only for the officer except for status changes.

Field	Value
UC-16	Add Candidate's CV (manual upload)
Primary Actor	HR Officer
Pre-conditions	The officer is viewing a specific candidate's application (or candidate record).
Post-conditions	The CV file is stored and linked to the candidate's profile.
Allowed Roles	HR_OFFICER

Table 24: Add Candidate's CV

Main Flow

1. [U] Officer clicks Upload CV on the candidate's application page.
2. [S] System displays a file-upload dialog (accepts PDF, max 5 MB).
3. [U] Officer selects a file and confirms.
4. [S] System validates the file type/size, stores it (versioned), and updates the candidate profile.
5. [S] System shows “CV uploaded successfully.”

Alternative Flows

- AF-01 – Invalid File System shows “Only PDF files up to 5 MB are allowed.”
- AF-02 – Storage Failure The system shows “Upload failed, please try again later.”

Business Rules

- BR-01 Every CV upload creates a new version; previous versions are retained for audit.
- BR-02 Uploaded CVs are scanned for viruses before storage.

Field	Value
UC-17	Rate Candidate's CV (HR rating)
Primary Actor	HR Officer

Pre-conditions	The officer is viewing a candidate's CV (uploaded manually or by candidate).
Post-conditions	Rating (1-5 stars) and optional comments are stored, visible to the hiring team.
Allowed Roles	HR_OFFICER

Table 25: Rate Candidate's CV (HR rating)

Main Flow

1. [U] Officer clicks Rate CV button.
2. [S] System presents a rating widget + comment field.
3. [U] Officer selects rating, adds comment (optional), submits.
4. [S] System validates rating range, persists the rating linked to the CV version.
5. [S] System shows a confirmation toast.

Alternative Flows – same as Candidate self-rating (invalid input, missing CV).

Business Rules – same as UC-10 (immutable records, average rating calculation).

Field	Value
UC-18	Track Candidate's CV (view rating history)
Primary Actor	HR Officer
Pre-conditions	The officer has access to the candidate's profile.
Post-conditions	The officer sees a chronological list of all ratings/comments for each CV version.
Allowed Roles	HR_OFFICER

Table 26: Track Candidate's CV (view rating history)

Main Flow

1. [U] Officer opens a candidate's profile and selects CV History.
2. [S] System retrieves all CV versions and associated ratings/comments.
3. [S] System renders a timeline: version date → rating → comment → download link.

Alternative Flows – none (read-only).

Business Rules

- BR-01 Ratings cannot be edited; a new rating creates a new entry.

- BR-02 Only HR Officers can see the rating history.

3.3.2.5 Admin

Field	Value
UC-19	View Admin Dashboard
Primary Actor	Admin
Pre-conditions	Authenticated admin with ADMIN role.
Post-conditions	Dashboard shows system health, user statistics, recent audit logs.
Allowed Roles	ADMIN

Table 27: View Admin Dashboard

Main Flow

1. [U] Admin selects Admin Dashboard.
2. [S] System aggregates: total users, active jobs, error logs, recent admin actions.
3. [S] System displays charts, key-metrics, and quick-action links (Add Account, System Settings).

Alternative Flows – none (read-only).

Business Rules

- BR-01 Dashboard refresh interval = 2 minutes.
- BR-02 Only users with ADMIN roles may view audit logs.

Field	Value
UC-20	View Account List
Primary Actor	Admin
Pre-conditions	Authenticated admin.
Post-conditions	Admin sees a searchable, filterable list of all user accounts (Candidate, HR Officer, Admin).
Allowed Roles	ADMIN

Table 28: View Account List

Main Flow

1. [U] Admin clicks Accounts in the side menu.
2. [S] System queries the User table and returns all records.

3. [S] System displays a table: Username, Email, Role, Status, Actions (View, Edit, Activate/Deactivate).

Alternative Flows

- AF-01 – No Accounts “No user accounts found.” – unlikely but handled.
- AF-02 – Filter by Role Admin selects a role filter; system re-queries and updates view.

Business Rules

- BR-01 Email addresses are globally unique.
- BR-02 Admin cannot delete their own account (self-protect).

Field	Value
UC-21	Add New Account
Primary Actor	Admin
Pre-conditions	Admin is authenticated; the role to be created is either HR_OFFICER or ADMIN (candidates self-register).
Post-conditions	A new user record exists with status ACTIVE (or INACTIVE if email verification is required).
Allowed Roles	ADMIN

Table 29: Add New Account

Main Flow

1. [U] Admin selects Add Account.
2. [S] System shows a form: Full Name, Email, Role (dropdown), Temporary Password, optional Department.
3. [U] Admin fills the fields and clicks Create.
4. [S] System validates (see BR-01-BR-04).
5. [S] System creates the user, sets the role, hashes the temporary password, and sends a welcome e-mail containing a password-reset link.
6. [S] System redirects to the Account List with a success toast.

Alternative Flows

- AF-01 – Email Duplicate The system shows “An account with that e-mail already exists.”
- AF-02 – Validation Errors Inline messages; no account is created.
- AF-03 – Email Delivery Failure The system logs the failure and provides a Resend option.

Business Rules

- BR-01 Roles can be ADMIN or HR_OFFICER only.
- BR-02 Temporary passwords must meet the same complexity as user passwords.
- BR-03 Admin must set the account status to ACTIVE unless the organization mandates email verification (then INACTIVE).
- BR-04 Admin actions are recorded in an audit trail (who, when, what).

Field	Value
UC-22	View/Update Account Detail
Primary Actor	Admin
Pre-conditions	Admin selects an existing account from the list.
Post-conditions	Account data (name, email, role, status) is updated; audit entry recorded.
Allowed Roles	ADMIN

Table 30: View/Update Account Detail

Main Flow

1. [U] Admin clicks Edit beside an account.
2. [S] System loads the account details into an editable form.
3. [U] Admin modifies allowed fields (e.g., role, status) and clicks Save.
4. [S] System validates changes (email cannot be changed, role change follows BR-01).
5. [S] System updates the record, logs the modification, and shows a confirmation.

Alternative Flows

- AF-01 – Attempt to Demote Last Admin The system blocks the change and shows “At least one admin must remain.”
- AF-02 – Validation Failure Inline errors; no update performed.

Business Rules

- BR-01 Email address is immutable after creation.
- BR-02 Role changes are limited to ADMIN ↔ HR_OFFICER; a candidate cannot be promoted here.
- BR-03 Deactivating an account invalidates all active sessions.

Field	Value
UC-23	Activate/Deactivate Account

Primary Actor	Admin
Pre-conditions	Admin views the account list (or detail page).
Post-conditions	Account status toggles between ACTIVE and INACTIVE; all sessions are revoked on deactivation.
Allowed Roles	ADMIN

Table 31: Activate/Deactivate Account

Main Flow

1. [U] Admin clicks the Activate/Deactivate toggle button for a target account.
2. [S] System updates the status column, revokes any existing sessions if deactivated, and logs the action.
3. [S] System shows a toast “Account X is now ACTIVE/INACTIVE.”

Alternative Flows

- AF-01 – Attempt to Deactivate Self The system prevents the action and displays “You cannot deactivate your own account.”
- AF-02 – System Error System displays “Operation failed; please retry.”

Business Rules

- BR-01 At least one ADMIN must stay ACTIVE at all times.
- BR-02 Deactivated accounts cannot log in until re-activated.
- BR-03 All authentication tokens for the account are invalidated immediately.

3.4 Functional Requirements

3.4.1 System Features

The functional requirements are organised by module.

Module	Feature
Authentication	Candidate self-registration with OTP email verification
Authentication	Sign in with email and password (JWT-based)
Authentication	Sign out (client-side JWT discard)
Authentication	Forgot / reset password via OTP email
Job Management	Create structured job posting (title, skills, experience, education, JD text)

Job Management	Edit OPEN job posting
Job Management	Close or delete job posting; auto-close on deadline
AI CV Processing	Bulk-upload PDF/DOCX CV files for a job posting
AI CV Processing	Auto-parse CV via LLM (extract name, skills, experience, education)
AI CV Processing	Auto-score candidate via LLM against job requirements
AI CV Processing	Auto-generate AI summary and recommendation label
HR Candidate Mgmt	View ranked candidate list with filters (score, experience, education)
HR Candidate Mgmt	View candidate detail - parsed profile, score breakdown, AI reasoning
HR Candidate Mgmt	Mark HR decision: SHORTLISTED / ON_HOLD / REJECTED
HR Candidate Mgmt	Download original CV file (access-controlled)
Candidate Portal	Public job listing and job detail (no sign-in required)
Candidate Portal	Self-apply with CV upload; triggers AI pipeline
Candidate Portal	Track own application status and view AI recommendation
Candidate Portal	Withdraw own application
Admin	Create, edit, activate/deactivate user accounts
Admin	View all users and global platform statistics
Dashboard	HR Officer dashboard - active jobs, candidates, decisions
Dashboard	Candidate and Admin dashboards - role-specific stats

Table 32: Functional Requirements by Module

3.4.2 Screen Inventory

a. Screen Flow

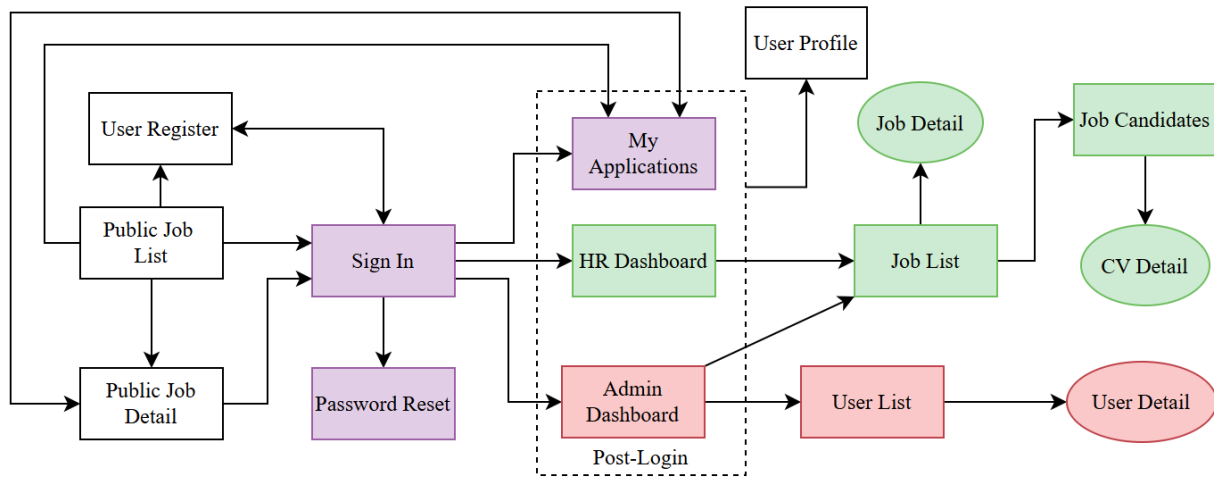


Figure 9: Screen Flow

b. Screen Description

SCR-ID	Screen Name	Actor	Description
SCR-01	Sign In	All(unauthenticated)	Allows any user to authenticate with email and password, receiving a JWT token <u>Entry Point</u> : Direct URL; any protected route redirect
SCR-02	User Register	Guest	Allows a new Candidate to create their own account; account requires OTP verification before activation. <u>Entry Point</u> : SCR-01 "Create account" link; SCR-10 "Apply" button (unauthenticated)
SCR-03	Password Reset	All	Allows a user to request a password reset email containing an OTP. Then, allows a user to set a new password using the OTP received by email. <u>Entry Point</u> : SCR-01 "Forgot password?" link
SCR-04	HR Dashboard	HR_Officer, Admin	Provides HR officers with a summary view of their active recruitment pipeline. <u>Entry Point</u> : Post sign-in redirect (HR Officer); nav sidebar
SCR-05	Job List	HR_Officer, Admin	Shows the HR_Officer a list of all their job postings with status and quick-action buttons. <u>Entry Point</u> : Nav sidebar

SCR-06	Job Detail	HR_Officer, Admin	Form for creating a new job posting or editing an existing OPEN one. <u>Entry Point</u> : SCR-05 "New Job" button; SCR-07 "Edit" button
SCR-07	Job Candidates	HR_Officer, Admin	Central workspace for HR officers: view job details, upload CVs, export. <u>Entry Point</u> : SCR-05 row click
SCR-08	CV Detail	HR_Officer, Admin	Review AI-ranked candidates, mark decisions and relevant notes. <u>Entry Point</u> : SCR-07 "View Detail" icon
SCR-09	Public Job List	All (incl. Guest)	Allows any visitor to browse all OPEN job postings. <u>Entry Point</u> : Direct URL; top navigation
SCR-10	Public Job Detail	All (incl. Guest)	Shows full job details to any visitor; allows signed-in Candidates to self-apply by uploading their CV. <u>Entry Point</u> : SCR-09 job card click
SCR-11	My Applications	Candidate	Shows a Candidate all their submitted applications with AI processing status, recommendation, and option to withdraw. <u>Entry Point</u> : Nav sidebar; post sign-in redirect (Candidate)
SCR-12	Admin Dashboard	Admin	Provides the Admin with a platform-wide overview of all users, jobs, and candidate processing stats. <u>Entry Point</u> : Post sign-in redirect (Admin); nav sidebar
SCR-13	User List	Admin	Show all user accounts with status and quick-action buttons. <u>Entry Point</u> : SCR-12; nav sidebar
SCR-14	User Detail	Admin	Form for creating a new account or editing existing one. <u>Entry Point</u> : SCR-13 "Create User" or action button
SCR-15	User Profile	All (authenticated)	Allows any signed-in user to view and update their profile information (full name, email, password) and avatar. <u>Entry Point</u> : Top-nav user dropdown => "Profile"

Table 33: Screen Description

3.5 Non-Functional Requirements

Category	Requirement	Target / Constraint
----------	-------------	---------------------

Performance	API response time	Under 2 seconds for any request with up to 50 concurrent users
Performance	AI processing time per CV	Under 30 seconds end-to-end (parse + score + summarise)
Performance	Export generation	Under 10 seconds
Performance	Maximum CV file size	5 MB per file (PDF or DOCX)
Security	Password storage	BCrypt hashing; plaintext passwords never stored
Security	Authentication	JWT tokens required for all protected endpoints; expired tokens rejected
Security	CV file access	Files served only through ownership-verified API endpoints; direct cloud URLs not exposed
Security	OTP lifecycle	OTP codes expire after a fixed period; used codes invalidated immediately
Security	CORS policy	API accepts requests only from configured frontend origin(s)
Reliability	AI processing resilience	Processing failure for one CV does not block other CVs; failed CV marked ERROR, pipeline continues
Usability	Modern browser support	Chrome 120+, Firefox 120+, Edge 120+; responsive layout down to 768 px width
Maintainability	API documentation	OpenAPI 3.0 specification auto-generated via SpringDoc; available at /swagger-ui
Deployment	Target environment	Single server deployment (Spring Boot JAR + React SPA via Nginx + PostgreSQL)

Table 34: Non Functional Requirements

3.6 Conceptual Data Model

3.6.1 Entity Relationship Diagram (ERD)

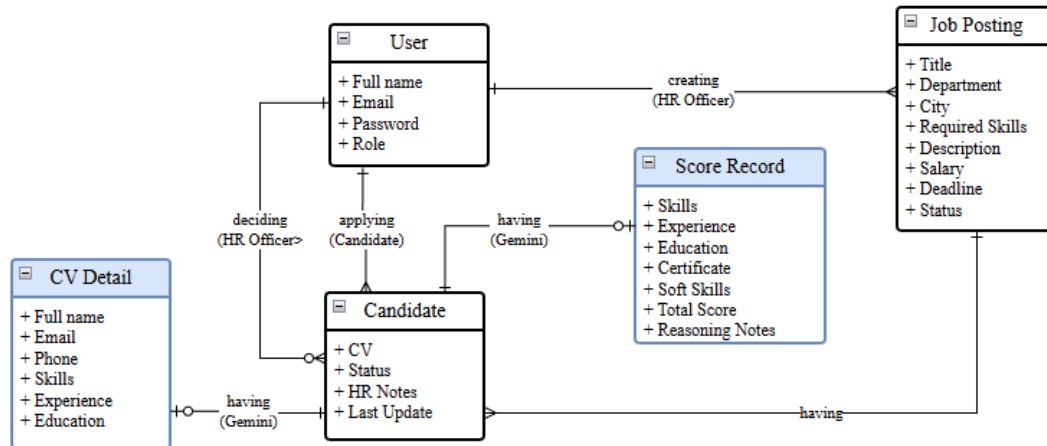


Figure 10: Entity Relationship Diagram

3.6.2 Entity Description

a. User

Field	Value
Purpose	Every person with an account in the system, regardless of role
Key business attributes	Full name, email address, role (ADMIN / HR_OFFICER / CANDIDATE), account status (active / inactive), avatar image URL
Business identity	Email addresses are unique across the entire system. Email is the login identifier.
Status / lifecycle	Active (normal operation) Inactive (deactivated by Admin; cannot sign in)

Table 35: User Description

b. Job Post

Field	Value
Purpose	A job opening created by an HR_Officer, visible to all visitors when OPEN
Key business attributes	Job title, department, city, job description text, required skills (text), minimum experience years, required education level (HIGH_SCHOOL / BACHELOR / MASTER / PHD), salary, application deadline, city, company logo URL
Business identity	No natural unique key - the system assigns an identifier. Multiple postings with the same title in the same department are permitted.

Field	Value
Status / lifecycle	OPEN (accepting applications) → CLOSED (not accepting applications; terminal). Auto-closes when the deadline passes.

Table 36: Job Post Description

c. Candidate

Field	Value
Purpose	A CV file submitted for a specific job, either by an HR_Officer on behalf of an applicant, or by a Candidate who self-applied
Key business attributes	Which job, original filename, file path (Clouinary URL), raw extracted text, AI processing status, error message (if failed), upload timestamp, which Candidate user (if self-applied)
Business identity	A Candidate user may submit at most one CV per job. HR bulk uploads have no uniqueness constraint per file.
Status / lifecycle	PENDING → PARSING → SCORING → SUMMARIZING → COMPLETED (terminal, success) / ERROR (terminal, failure)

Table 37: Candidate Description

d. CV Detail

Field	Value
Purpose	Structured data extracted from the CV by the AI parsing step
Key business attributes	Full name, email, phone, skills list, years of professional experience, education level
Business identity	Exactly one CV Detail per Candidate record. Created during the PARSING phase; never updated afterward.
Status / lifecycle	Immutable once created - represents the AI's extraction at time of processing

Table 38: CV Detail Description

e. Score Record

Field	Value
Purpose	The AI-computed weighted scores for a candidate against their job's requirements
Key business attributes	Total score (0–100), skills score, experience score, education score, certification score, soft skills score, reasoning note
Business identity	Exactly one Score Record per Candidate record. Created during the SCORING phase; read-only for all users.
Status / lifecycle	Immutable once created - the score reflects the AI's assessment at time of processing and cannot be manually edited

Table 39: Score Record Description

3.7 AI Scoring Algorithm

The scoring algorithm is the core intellectual contribution of the system design. Each candidate is evaluated against the job's requirements across five dimensions:

Dimension	Weight	Scoring Criteria
Skills Match	35%	Overlap between the candidate's extracted skills and the job's required skills list; partial credit for adjacent skills
Professional Experience	30%	Candidate's years of experience relative to the minimum required; domain-relevance assessed by LLM
Education Level	15%	Candidate's highest qualification relative to the required education level (HIGH_SCHOOL / BACHELOR / MASTER / PHD)
Certifications	10%	Presence of relevant professional certifications mentioned in the CV
Soft Skills	10%	Inference of communication, leadership, or teamwork indicators from CV text

Table 40: AI Scoring Weight Distribution

The total score (0–100) maps to three recommendation labels: `STRONG_MATCH` (80–100), `POTENTIAL_MATCH` (60–79), and `NOT_RECOMMENDED` (0–59). The LLM is instructed to return a structured JSON response for each scoring call, ensuring that all five dimension scores and their reasoning can be stored and displayed to HR Officers transparently.

IV/ MATERIALS AND METHODS

4.1 Technology Stack

The technology choices for RecruitIQ were driven by three principles: suitability for the functional requirements (particularly AI integration and file processing), practical deployability on a single-server production environment, and team familiarity. The selected stack is summarised in the table below.

Layer	Technology	Version	Rationale
-------	------------	---------	-----------

Backend framework	Spring Boot	3.2.3	Mature REST API framework; embedded Tomcat; strong ecosystem for JPA, security, and async processing
Backend language	Java	17 (LTS)	Long-term support; compatible with Spring Boot 3.x; strong type safety
Frontend framework	React + TypeScript	19 / 5.x	Component-based SPA with strong typing; rich ecosystem for complex UI interactions
Frontend build	Vite	6.x	Fast HMR dev server; optimised production builds
Database	PostgreSQL	16.x	Robust relational database; strong JSON support for future extensibility
ORM	Spring Data JPA / Hibernate	6.x	Reduces boilerplate; JPQL for complex queries
Authentication	JWT	0.12.3	Stateless; suitable for decoupled SPA + API architecture
PDF extraction	Apache PDFBox	3.0.2	Best-in-class for structured text extraction from PDF files
DOCX extraction	Apache POI	5.2.5	Standard library for Office Open XML document parsing
PDF export	iText7	8.0.4	Generates formatted PDF candidate reports
File storage	Cloudinary	1.36.0	Cloud storage for CV files; CDN delivery; eliminates server disk concerns
AI / LLM	Google Gemini API	gemini-2.5-flash	Generous free tier; strong JSON output capability; fallback model available
Email	Spring Boot Starter Mail	managed	SMTP integration for OTP and password reset emails
API docs	SpringDoc OpenAPI	2.3.0	Auto-generated Swagger UI from controller annotations

Table 41: Technology Stack

4.2 System Architecture

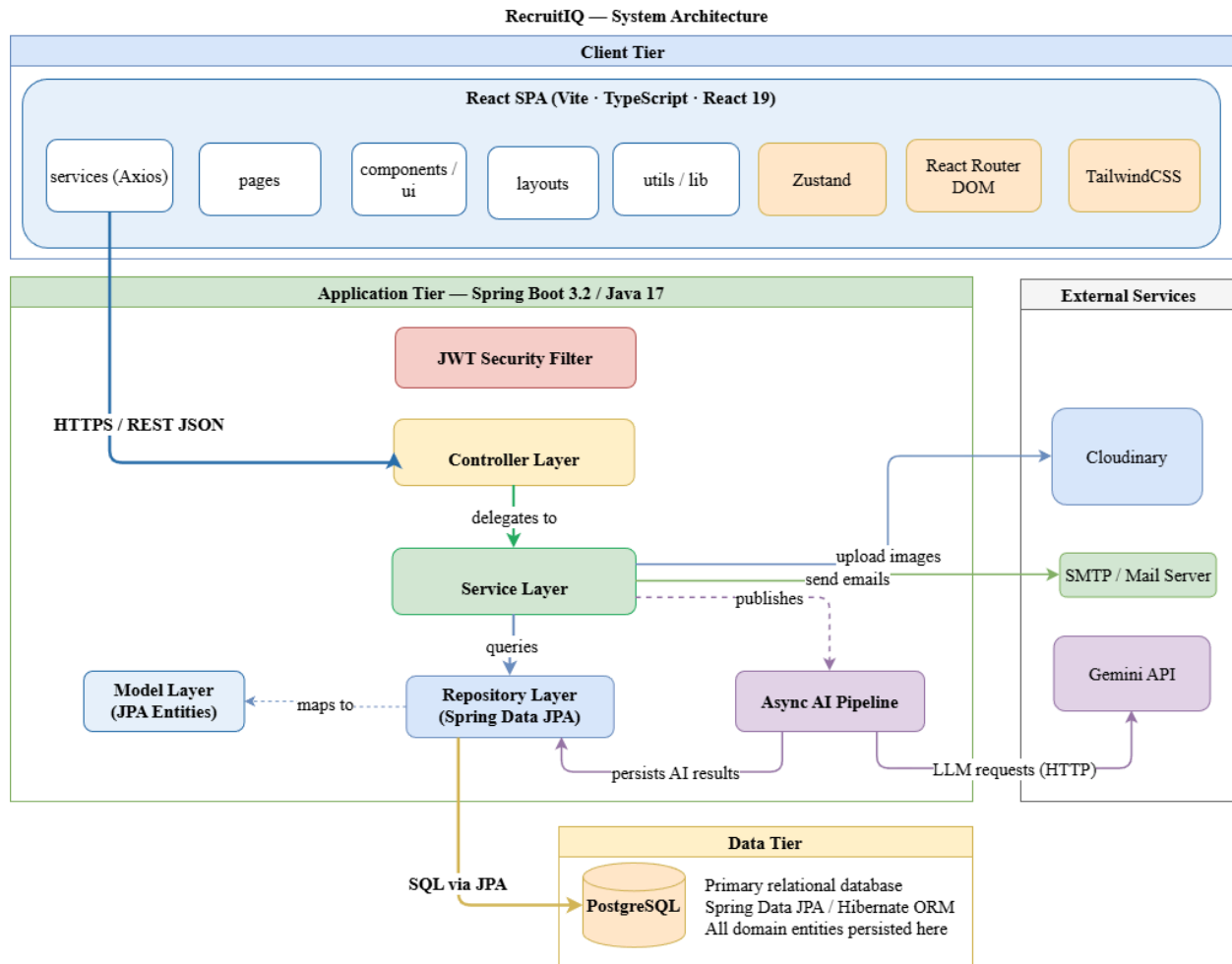


Figure 11: System Architecture

The system is split into four tiers that reflect both physical deployment boundaries and logical separation of concerns.

Tier	Technology	Responsibility
Client Tier	React 19 / TypeScript / Vite	Renders the UI in the browser; communicates with the backend exclusively over HTTPS REST
Application Tier	Spring Boot 3.2 / Java 17	Hosts all business logic, security, and data access; the only process with database credentials
Data Tier	PostgreSQL	Durable storage for all domain entities; accessed only by the Application Tier
External Services	· Cloudinary · Gemini API	Third-party cloud services consumed by the Application Tier for storage, AI, and email

Tier	Technology	Responsibility
	· SMTP	

Table 42: System architecture

4.2.1 Client Tier

- Everything that runs inside the user's web browser. The entire frontend is a single-page application (SPA) bundled by Vite. No page navigation causes a full server reload - the browser holds the application state between route changes and communicates with the backend through a typed HTTP service layer.

1. React SPA (Vite · TypeScript · React 19)

- The top-level container for the frontend application. Vite serves and bundles the code; TypeScript enforces type safety across the entire client codebase. The SPA initialises once on page load and thereafter manages its own routing, state, and data fetching without server-side rendering.

2. pages

- Route-level components, each mapped one-to-one to a URL. Pages are the only layer that initiates data fetching - they call service functions, hold the resulting state locally, and compose smaller components to render the view. Pages are grouped by role (auth, admin, hr, portal) so that role-based access control at the routing level has a clear structural home.

- Connects to: services (Axios) (outbound data requests), components / ui (rendering), layouts (visual shell).

3. components / ui

- Reusable React components shared across pages. Divided into two sub-groups: structural helpers (ProtectedRoute, UserMenuDropdown) that enforce auth guards and provide navigation widgets, and primitive UI building blocks (Button, Card, Input, ImageUpload) used everywhere in the application. Nothing in this package fetches data or holds application-level state.

- Connects to: Zustand (reads auth/user state for ProtectedRoute and UserMenuDropdown), TailwindCSS (all styling).

4. layouts

- Role-specific shell wrappers that provide the persistent navigation chrome surrounding each section of the application - header, sidebar, and footer. A layout renders its frame and delegates the active page content to a React Router outlet slot. Separating shell from page ensures page components never conditionally show or hide navigation elements. Connects to: pages (renders active page via outlet), Zustand (reads session to personalise navigation).

5. services (Axios)

- The frontend's entire HTTP layer. A base api.ts module creates a single Axios instance, attaches the JWT Authorization header from Zustand on every request, and handles 401 responses globally (clears the session and redirects to login). All other service modules (authService, jobService, hrService, etc.) are thin wrappers around this instance, each

owning the typed endpoint functions for one domain area. No page or component ever imports Axios directly.

- Connects to: Application Tier / Controller Layer via HTTPS / REST JSON (the primary cross-tier boundary), Axios (underlying HTTP client).

6. utils / lib

- Stateless, framework-agnostic helper modules. Contains pure form-validation functions used by the hr pages (job create/edit) and general-purpose utilities such as CSS class merging shared across the rest of the application. Nothing in this package renders UI or makes network calls.

7. Zustand

- Client-side global state store. Holds the authenticated user object, assigned role, and JWT token so any component can read session state without prop-drilling. It is the single source of truth for authentication status on the client: ProtectedRoute reads it to gate routes, layouts reads it to personalise the header, and services/api.ts reads it to inject the Bearer token.

8. React Router DOM

- Declarative client-side routing library. Defines the URL-to-component mapping in App.tsx, provides <Outlet> for layouts to embed pages, and exposes navigation hooks used by pages and auth flows to redirect programmatically after login, logout, or unauthorised access.

9. TailwindCSS

- Utility-first CSS framework. All visual styling across pages, layouts, and components is applied through class names directly on JSX elements - there are no separate per-component CSS or SCSS files. Keeps styling co-located with markup and eliminates dead CSS.

4.2.2 Application Tier

- The Spring Boot process that runs on the server. It is the only tier with credentials for the database and external services. All business rules, security enforcement, and data transformations live here. The tier is internally layered - requests flow top-to-bottom through the filter, controller, service, and repository layers - and one asynchronous sub-system (the AI pipeline) runs independently of the request thread.

1. JWT Security Filter

- Package: com.recruitiq.security Components: JwtAuthenticationFilter, JwtUtil, CustomUserDetailsService

- A OncePerRequestFilter that sits in front of every controller in the application. On each inbound request it extracts the Bearer token from the Authorization header, validates its signature and expiry using JwtUtil, loads the matching user record via CustomUserDetailsService, and populates the Spring Security context with the caller's identity and role. Requests without a valid token are rejected before reaching any controller. The security support package (bottom row) supplies these beans; SecurityConfig (inside config) registers them into the filter chain.

- Connects to: Controller Layer (guards all inbound requests), security support package (bean definitions), com.recruitiq.repository (loads user by email for authentication).

2. Controller Layer

- Package: com.recruitiq.controller Components: AuthController, UserController, AdminController, JobController, CandidateController, CandidatePortalController, HRDashboardController, CityController, FileController, ApiController
- The HTTP boundary of the application. Each controller declares REST route mappings (@RestController, @RequestMapping), validates inbound request bodies via Bean Validation (@Valid), and immediately delegates every operation to the service layer - no business logic lives here. Responses are shaped by the dto package before being serialised to JSON. HTTP status codes are decided at this layer.
- Connects to: Service Layer (sync delegation, primary downstream), dto (reads inbound DTOs, writes outbound DTOs), validation (Bean Validation triggered on @Valid-annotated parameters).

3. Service Layer

- Package: com.recruitiq.service Components: AuthService, UserService, JobService, CandidateService, CityService, FileProcessingService, FileDownloadService, ImageStorageService, MailService, ExportService
- The business logic layer and the heart of the application. Services orchestrate repository queries, enforce domain rules (e.g. only one shortlist decision per candidate/job pair, deadline validation), and coordinate cross-cutting work such as sending emails or uploading images. They are the only layer permitted to call repositories directly. After processing, services use the mapper package to convert entities to DTOs before returning data to controllers. FileProcessingService parses uploaded CV documents and publishes a CandidateUploadedEvent to trigger the async AI pipeline.
- Connects to: Repository Layer (sync data access), mapper (entity-to-DTO conversion), event (publishes CandidateUploadedEvent), Cloudinary (image uploads via ImageStorageService), SMTP / Mail Server (email delivery via MailService).

4. Async AI Pipeline

- Package: com.recruitiq.service.ai
- A self-contained sub-system that analyses candidate CVs using a large-language model. It is fully decoupled from the upload HTTP request through Spring's event mechanism - the upload endpoint returns immediately and the pipeline processes the CV on a dedicated async thread pool. The pipeline runs three sequential stages (parse → score → summarise) and persists each stage's result independently so partial results are available even if a later stage fails.
- Connects to: event (receives CandidateUploadedEvent), Repository Layer (persists AI results to AiSummaryRepo, ParsedProfileRepo, ScoreRecordRepo), Gemini API (LLM calls via LlmApiClient), ai (PromptConstants) (prompt templates).
 - a. AiProcessingListener**
 - Entry point of the async pipeline. Annotated with @TransactionalEventListener so it fires after the publishing transaction commits, guaranteeing the candidate record is fully persisted before AI processing begins. Hands off immediately to AiProcessingService.

b. **AiProcessingService**

- Orchestrates the three-stage pipeline. Calls ParseService, ScoreService, and SummaryService in order and persists each result through the repository layer. Handles stage-level error isolation so a failure in scoring does not prevent summary generation.

c. **ParseService**

- Stage 1. Sends the raw CV text to the Gemini API with a structured extraction prompt and parses the JSON response into a ParsedProfile entity (name, skills, experience, education, contact details).

d. **ScoreService**

- Stage 2. Sends the parsed profile and the target job description to the Gemini API with a scoring prompt. Returns a numeric relevance score and a brief rationale, stored as a ScoreRecord.

e. **SummaryService**

- Stage 3. Sends the parsed profile to the Gemini API with a summarisation prompt. Returns a concise plain-text summary of the candidate's background, stored as an AiSummary.

f. **LlmApiClient**

- Low-level HTTP client for the Gemini API. Constructs the request payload, injects the API key from application properties, sends the POST request via RestTemplate, and parses the response. Injects prompt text from PromptConstants. All three AI services funnel their LLM calls through this single client.

5. Repository Layer

- Package: com.recruitiq.repository Components: UserRepo, CandidateRepo, JobRepo, - CityRepo, AiSummaryRepo, ParsedProfileRepo, PasswordResetTokenRepo, ScoreRecordRepo, ShortlistRepo

Spring Data JPA repository interfaces. Each interface is the sole point of database access for one aggregate root. Queries are declared as derived-name methods or @Query annotations - no SQL string appears anywhere else in the application. Hibernate translates these into SQL at runtime and manages the connection pool. The repository layer is called by both the synchronous Service Layer and the async AI Pipeline.

- Connects to: Model Layer (maps query results to entity objects), PostgreSQL (executes SQL via Hibernate/JPA).

6. Model Layer

- Package: com.recruitiq.model Entities: User, Candidate, Job, City, AiSummary, ParsedProfile, PasswordResetToken, ScoreRecord, Shortlist

- JPA entity classes annotated with @Entity and mapped to PostgreSQL tables. These are pure data containers - they carry no business logic. Lombok eliminates boilerplate getters, setters, and constructors. Entities never cross the service boundary outward; the mapper layer converts them to DTOs before they are returned to controllers.

- Connects to: Repository Layer (loaded and persisted by repositories), PostgreSQL (table-level representation).

4.2.3 Data Tier

1. PostgreSQL

- The application's sole durable data store. All nine domain entities are persisted here in a normalised relational schema managed by Hibernate. The database is never accessed directly by the Client Tier or External Services - only the Application Tier holds connection credentials. Schema migrations are handled at startup via Hibernate DDL or an external migration tool.
- Receives: SQL queries from Repository Layer via the JPA/Hibernate ORM.

2. External Services

- Third-party cloud services consumed by the Application Tier at runtime. All calls are outbound from the server - the Client Tier never contacts these services directly.

a. Cloudinary

- Cloud-hosted image and file CDN. Used by ImageStorageService to upload and store profile photos and company images. Cloudinary processes the upload, returns a publicly accessible CDN URL, and handles image optimisation and delivery. The returned URL is persisted on the entity record and served to the client in API responses - the Spring Boot process does not serve image bytes.
- Called by: Service Layer (ImageStorageService) on profile/image upload requests.

b. Gemini API (google/gemini-2.5-flash)

- Google's large-language model API. Used exclusively by the async AI pipeline for three tasks: extracting structured data from raw CV text (parse), scoring a candidate's profile against a job description (score), and writing a concise plain-text summary of a candidate (summary). The model is gemini-2.5-flash, configured via llm.api.url and llm.api.key application properties. All calls are made by LlmApiClient using prompt templates from PromptConstants.
- Called by: LlmApiClient (inside the AI Pipeline sub-system) after each CV upload event.

c. SMTP / Mail Server

- Transactional email delivery. Used by MailService through Spring's JavaMailSender abstraction, so the underlying mail provider can be swapped without code changes. Sends two categories of email: password-reset links (triggered by AuthService during the forgot-password flow) and application-status notifications to candidates when HR officers make shortlist decisions.
- Called by: Service Layer (MailService) for password reset and candidate notification emails.

4.3 AI Processing Pipeline Design

The AI processing pipeline is event-driven and fully asynchronous. When a CV file is uploaded (either by an HR Officer or by a Candidate self-applying), the following sequence executes:

- File ingestion: The file is validated (format, size), uploaded to Cloudinary for persistent storage, and a Candidate record is created in the database with status PENDING.
- Event publication: A CandidateUploadedEvent Spring application event is fired, carrying the candidate record ID.
- Async listener: AiProcessingListener (annotated with @EventListener) receives the event and invokes AiProcessingService.processCandidateAsync() on a dedicated thread pool (@Async with taskExecutor).
- Phase 1 - Parsing: ParseService extracts raw text from the file (PDFBox for PDF, Apache POI for DOCX) and calls the Gemini API with a structured parsing prompt. The LLM returns a JSON object containing the candidate's name, email, phone, skills list, years of experience, education level, and work history summary. The result is stored as a ParsedProfile entity and the candidate status is advanced to SCORING.
- Phase 2 - Scoring: ScoreService constructs a scoring prompt combining the job's JD text, required skills, minimum experience, and required education with the parsed candidate profile. The LLM returns a structured JSON object with five dimension scores (0–100), reasoning for each, a total weighted score, and lists of identified strengths and gaps. The result is stored as a ScoreRecord entity and the candidate status is advanced to SUMMARIZING.
- Phase 3 - Summarisation: SummaryService calls the Gemini API to generate a three-to-four sentence narrative assessment and determine the recommendation label based on the total score. The result is stored as an AiSummary entity and the candidate status is advanced to COMPLETED.
- Error handling: If any phase fails due to an LLM API error, network timeout, or malformed response, the candidate status is set to ERROR with an error message. Other candidates in the batch continue processing independently.

4.4 Database Design

4.4.1 Database Schema

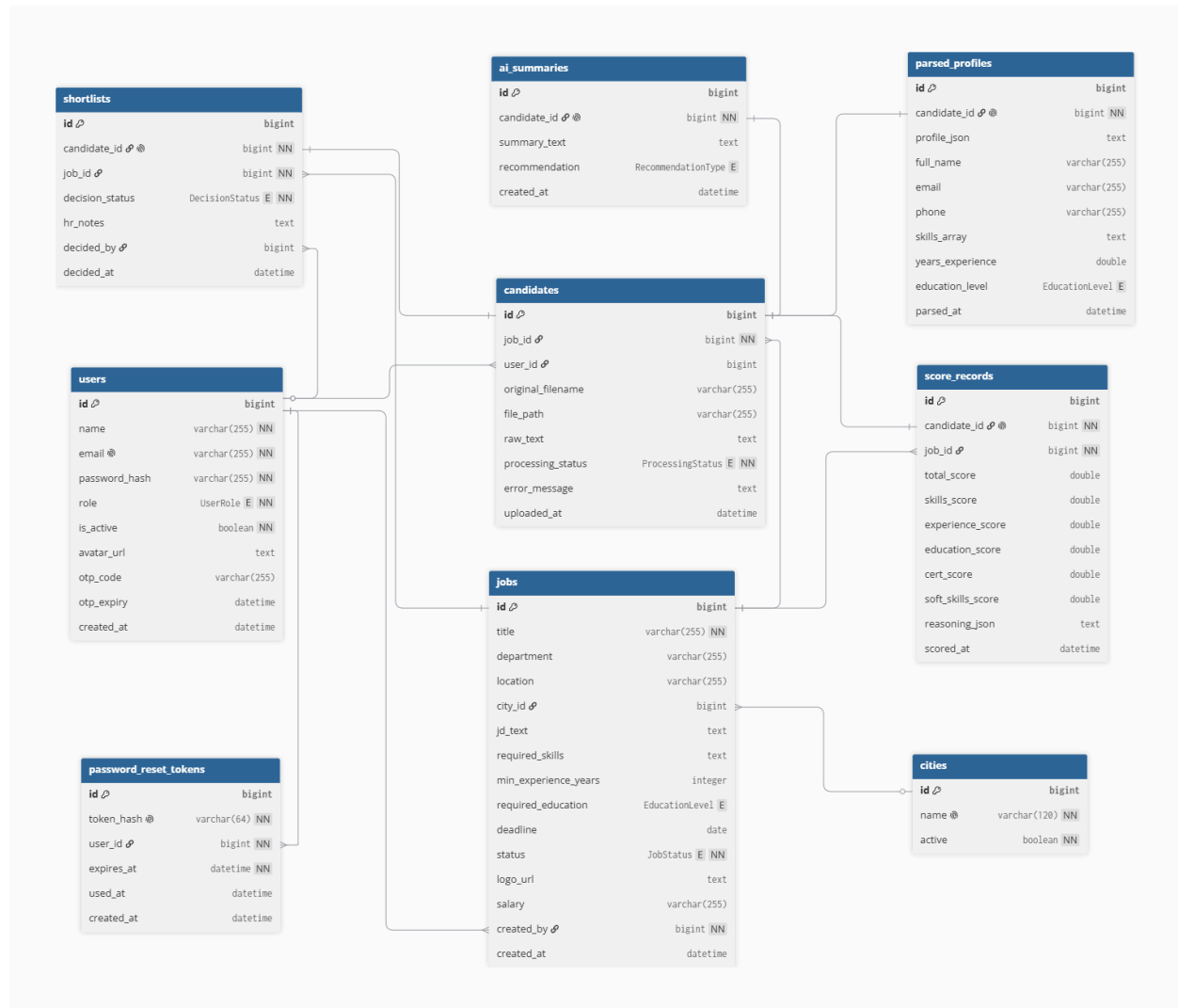


Figure 12: Database Schema

4.4.2 Database Schemas Description

The database schema consists of eight tables. The primary entities are: users, jobs, candidates (CV submissions), parsed_profiles, score_records, ai_summaries, shortlists, and cities (reference data). Key design decisions include:

- Separation of AI outputs into three tables (parsed_profiles, score_records, ai_summaries) rather than a single wide table. This aligns with the pipeline's three phases and enables querying only the data needed for a given UI view.

- Immutability of AI data: No update endpoints exist for parsed_profile, score_record, or ai_summary records. This preserves the integrity of the AI's assessment at the time of processing.
- Shortlist mutability: The shortlists table is designed for frequent updates - HR Officers routinely revise decisions - and includes an updated_at timestamp and decided_by FK to track accountability.
- Cloudinary URLs in the database: CV file paths are stored as Cloudinary URLs. Files are never served directly from the database or the application server filesystem.

4.4.3 Data Dictionary

4.4.3.1 Table: users

Corresponds to User.java. Stores system credentials and profiles for Administrators, HR Officers, and Candidates.

Column Name	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each user.
name	VARCHAR(255)	NOT NULL	The full name of the user.
email	VARCHAR(255)	NOT NULL, UNIQUE	The user's email address, used for login and notifications.
password_hash	VARCHAR(255)	NOT NULL	The securely hashed password of the user.
role	VARCHAR(255)	NOT NULL	System role defining user permissions. Allowed values: ADMIN, HR_OFFICER, CANDIDATE.
is_active	BOOLEAN	NOT NULL, DEFAULT true	Flag indicating whether the account is active.
avatar_url	TEXT	NULL	URL pointing to the user's profile picture or avatar.
otp_code	VARCHAR(255)	NULL	One-Time Password (OTP) used for password resets or verification.

otp_expiry	TIMESTAMP	NULL	Expiration timestamp for the generated OTP code.
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIME STAMP	The timestamp when the user account was created. Automatically generated and non-updatable.

Table 43: User Data

4.4.3.2 Table password_reset_tokens

Corresponds to PasswordResetToken.java. Manages secure tokens generated when users request a password recovery.

Column Name	SQL Data Type	Constraints	Description / Business Logic
id	BIGINT	PK, Auto Increment	Unique identifier for each user.
token_hash	VARCHAR(64)	UNIQUE, NOT NULL	Hashed value of the password reset token for security.
user_id	BIGINT	FK (users.id), NOT NULL	References the user requesting the password reset.
expires_at	DATETIME	NOT NULL	The strict deadline timestamp for token expiration.
used_at	DATETIME	NULL	Records the exact time the token was consumed.
created_at	DATETIME	NULL	Timestamp of the password reset request execution.

Table 44: Password Reset Tokens Data

4.4.3.3 Table cities

Corresponds to City.java. Master lookup catalog for geographical indexing of job posts.

Column Name	SQL Data Type	Constraints	Description / Business Logic
id	BIGINT	PK, Auto Increment	Unique identifier for the city.
name	VARCHAR(120)	UNIQUE, NOT NULL	Name of the city or province (e.g., Hanoi, HCMC).

active	BOOLEAN	NOT NULL, DEFAULT true	Toggle switch to enable or disable this location across job filters.
--------	---------	---------------------------	--

Table 45: City Data

4.4.3.4 Table jobs

Corresponds to Job.java. Houses details for all employment opportunities created by HR personnel.

Column Name	SQL Data Type	Constraints	Description / Business Logic
id	BIGINT	PK, Auto Increment	Unique identifier for the job vacancy.
title	VARCHAR(255)	NOT NULL	Job title (e.g., Senior Java Developer).
department	VARCHAR(255)	NULL	Division or organizational unit (e.g., Engineering, HR).
location	VARCHAR(255)	NULL	Specific office or room address where the job is based.
city_id	BIGINT	FK (cities.id), NULL	Linked city reference for localized queries.
jd_text	TEXT	NULL	Full textual Job Description containing roles and tasks.
required_skills	TEXT	NULL	Key technical or soft skills needed (utilized by the AI engine).
min_experience_years	INT	NULL	Minimum number of years of experience requested.
required_education	VARCHAR(50)	NULL	Mandatory academic degree target: HIGH_SCHOOL, BACHELOR, MASTER, PHD.
deadline	DATE	NULL	Application closing date threshold (LocalDate).
status	VARCHAR(50)	NOT NULL, DEFAULT 'OPEN'	Hiring cycle state: OPEN or CLOSED.
logo_url	TEXT	NULL	Custom icon/brand image path assigned to the posting.

salary	VARCHAR(255)	NULL	Salary bracket information string displayed to candidates.
created_by	BIGINT	FK (users.id), NOT NULL	References the specific HR Officer who owns the post.
created_at	DATETIME	NULL	Timestamp recording when the vacancy went live.

Table 46: Job Data

4.4.3.5 Table candidates

Corresponds to Candidate.java. The transactional centerpiece linking raw resume files to a targeted Job vacancy.

Column Name	SQL Data Type	Constraints	Description / Business Logic
id	BIGINT	PK, Auto Increment	Unique identifier for the candidate's application ticket.
job_id	BIGINT	FK (jobs.id), NOT NULL	References the job vacancy the profile is applying to.
user_id	BIGINT	FK (users.id), NULL	References the candidate's user account (if submitted via candidate portal).
original_filename	VARCHAR(255)	NULL	Name of the original document uploaded (e.g., JohnDoe_CV.pdf).
file_path	VARCHAR(255)	NULL	Target storage directory address on the server file system.
raw_text	TEXT	NULL	Complete unformatted text pulled from the file wrapper.
processing_status	VARCHAR(50)	NOT NULL, DEFAULT 'PENDING'	Pipeline stage of the parser background worker: PENDING, PROCESSING, COMPLETED, ERROR.
error_message	TEXT	NULL	Log tracing details if stack exceptions crash parsing.
uploaded_at	DATETIME	NULL	Chronological entry marking file ingress.

Table 47: Candidate Data

4.4.3.6 Table: parsed_profiles

Corresponds to ParsedProfile.java. Structurally mirrors the profile fields extracted out of unformatted resume texts by LLMs.

Column Name	SQL Data Type	Constraints	Description / Business Logic
id	BIGINT	PK, Auto Increment	Unique identifier for the parsed profile.
candidate_id	BIGINT	FK (candidates.id), UNIQUE, NOT NULL	Strict 1:1 relationship pointing back to the core applicant submission.
profile_json	TEXT	NULL	Complete structured model backup preserved in raw JSON format.
full_name	VARCHAR(255)	NULL	Isolated full legal name of the candidate.
email	VARCHAR(255)	NULL	Contact email isolated directly from the applicant's CV page.
phone	VARCHAR(255)	NULL	Extracted phone number contact string.
skills_array	TEXT	NULL	Comma-separated or array format listing matching technical tag words.
years_experience	DOUBLE	NULL	Computed years of domain experience extracted from employment history timelines.
education_level	VARCHAR(50)	NULL	Highest verified scholastic milestone found: HIGH_SCHOOL, BACHELOR, MASTER, PHD.
parsed_at	DATETIME	NULL	Timestamp logging when the AI extraction process concluded successfully.

Table 48: Parsed_profiles Data

4.4.3.7 Table: score_records

Corresponds to ScoreRecord.java. Numerical matrix evaluations quantifying overall fit calculations between an applicant and a job post.

Column Name	SQL Data Type	Constraints	Description / Business Logic
id	BIGINT	PK, Auto Increment	Unique identifier for the score sheet.
candidate_id	BIGINT	FK (candidates.id), UNIQUE, NOT NULL	Strict 1:1 bond mapping score metrics to one unique resume payload.
job_id	BIGINT	FK (jobs.id), NOT NULL	Relational hook used for performant indexing inside job matrices.
total_score	DOUBLE	NULL	Aggregated, normalized matrix score output.
skills_score	DOUBLE	NULL	AI evaluation rating technical matching compatibility.
experience_score	DOUBLE	NULL	Rating computed based on tenure duration matches.
education_score	DOUBLE	NULL	Rating based on academic qualification matching parameters.
cert_score	DOUBLE	NULL	Extracted bonus weighting factors for relevant credentials.
soft_skills_score	DOUBLE	NULL	AI parsing metrics auditing operational behavioral patterns.
reasoning_json	TEXT	NULL	Detailed structural textual breakdown explaining the scoring rationale from the LLM.
scored_at	DATETIME	NULL	Execution checkpoint detailing when scoring algorithms ran.

Table 49: Score_records Data

4.4.3.8 Table: ai_summaries

Corresponds to AiSummary.java. Qualitatively highlights quick summary parameters to streamline manual HR scanning.

Column Name	SQL Data Type	Constraints	Description / Business Logic
id	BIGINT	PK, Auto Increment	Unique identifier for the AI synthesis file.

candidate_id	BIGINT	FK (candidates.id), UNIQUE, NOT NULL	Strict 1:1 map link to a specific incoming application sheet.
summary_text	TEXT	NULL	Condensed editorial assessment outlining an applicant's strong and weak points.
recommendation	VARCHAR(50)	NULL	Automated alignment verdict tag: STRONG_MATCH, POTENTIAL_MATCH, NOT_RECOMMENDED.
created_at	DATETIME	NULL	Timestamp tracking the generation of the assessment text.

Table 50: Ai_summaries Data

4.4.3.9 Table shortlists

Corresponds to Shortlist.java. Tracks the pipeline status and hiring decisions made by HR Officers.

Column Name	SQL Data Type	Constraints	Description / Business Logic
id	BIGINT	PK, Auto Increment	Unique identifier for the decision record.
candidate_id	BIGINT	FK (candidates.id), UNIQUE, NOT NULL	1:1 operational bond tracking a candidate's status within the funnel.
job_id	BIGINT	FK (jobs.id), NOT NULL	Relational foreign key reference to filter applicants by Job.
decision_status	VARCHAR(50)	NOT NULL, DEFAULT 'PENDING'	Status state: PENDING, SHORTLISTED, ON_HOLD, REJECTED.
hr_notes	TEXT	NULL	Discretionary text logs added manually by review officers.
decided_by	BIGINT	FK (users.id), NULL	References the HR user account who authorized the selection phase state.
decided_at	DATETIME	NULL	Timestamp capturing when the decision was finalized.

Table 51: Shortlists Data

4.5 Security Design

The security architecture is built around stateless JWT authentication, role-based access control enforced by Spring Security, and several application-layer data protection measures:

- **JWT filter:** `JwtAuthenticationFilter` intercepts every request, extracts the Bearer token from the Authorization header, validates the signature and expiry, loads the corresponding `UserDetails` via `CustomUserDetailsService`, and sets the `SecurityContextHolder`. Requests without a valid token proceed without authentication context and are rejected by protected route rules.
- **Role-based endpoint protection:** `SecurityConfig` defines path-level rules. HR-side endpoints (`/api/candidates/**`, `/api/jobs/**`) require `ROLE_HR_OFFICER` or `ROLE_ADMIN`. Admin endpoints (`/api/admin/**`) require `ROLE_ADMIN`. Portal endpoints (`/api/portal/jobs`, `/api/portal/jobs/**`) are permit-all for GET requests; POST requests require `ROLE_CANDIDATE`.
- **Method-level ownership checks:** Service layer methods verify that the authenticated user owns the resource being accessed (e.g., a Candidate cannot view another Candidate's application; an HR Officer cannot access another officer's job postings through portal APIs).
- **CV file access control:** Cloudinary stores CV files under access-controlled paths. The application generates short-lived or role-verified download URLs on request. Direct Cloudinary public URLs are not exposed in any API response.
- **Password security:** All passwords are hashed with BCrypt before storage. Plaintext passwords are never logged or returned in API responses.
- **OTP security:** OTP codes have a configured expiry window. Used OTPs are invalidated immediately to prevent replay attacks.

4.6 REST API Design

The REST API follows standard conventions with resource-oriented URL paths and HTTP method semantics. Selected key endpoints are listed below:

Module	Method	Path	Role
Auth	POST	/api/auth/register	Public
Auth	POST	/api/auth/verify-otp	Public
Auth	POST	/api/auth/login	Public
Auth	POST	/api/auth/forgot-password	Public

Auth	POST	/api/auth/reset-password	Public
Jobs	GET	/api/jobs	HR_Officer, Admin
Jobs	POST	/api/jobs	HR_Officer, Admin
Jobs	PUT	/api/jobs/{id}	HR_Officer, Admin
Jobs	POST	/api/jobs/{id}/close	HR_Officer, Admin
Candidates	POST	/api/candidates/{jobId}/upload	HR_Officer, Admin
Candidates	GET	/api/candidates/job/{jobId}	HR_Officer, Admin
Candidates	GET	/api/candidates/{id}	HR_Officer, Admin
Candidates	PUT	/api/candidates/{id}/decision	HR_Officer, Admin
Candidates	GET	/api/candidates/{id}/download-cv	HR_Officer, Admin
Candidates	GET	/api/candidates/job/{jobId}/export	HR_Officer, Admin
Portal	GET	/api/portal/jobs	Public
Portal	POST	/api/portal/my-applications	Candidate
Portal	GET	/api/portal/my-applications	Candidate
Portal	DELETE	/api/portal/my-applications/{id}	Candidate (own)
Admin	GET	/api/admin/users	Admin
Admin	POST	/api/admin/users	Admin
Admin	PUT	/api/admin/users/{id}	Admin
Admin	POST	/api/admin/users/{id}/toggle-status	Admin

Table 52: Key REST API Endpoints

V/ RESULTS AND DISCUSSION

5.1 Results

5.1.1 Implemented System Overview

RecruitIQ was implemented as a complete, deployable web application. The frontend comprises 17 distinct screens distributed across four role-specific layout contexts (HR, Candidate Portal, Admin, and Authentication), supporting the full HR-led screening workflow, the candidate self-application workflow, and the administrative user-management workflow. The AI processing pipeline, the system's core technical contribution, was implemented as a fully asynchronous, event-driven sequence using Spring's native `@Async` and `@EventListener` mechanisms, requiring no external message broker.

An introduction paragraph should always precede a table or figure presenting results - never start a results section with a table or figure alone. The remainder of this section follows that convention, beginning with the unit-test outcomes.

5.1.2 Unit and Integration Test Results

Service	Test Cases	Pass	Fail
AuthService	18	18	0
JobService	14	14	0
CandidateService	22	22	0
AiProcessingService	12	12	0
ExportService	8	8	0
UserService	10	10	0
Total	84	84	0

Table 53: Test Results

Integration tests covering all 22 protected API endpoints confirmed correct enforcement of JWT validation (401 returned for expired, malformed, or missing tokens across 25 endpoint/role combinations), role-based access control (HR Officer and Candidate roles correctly blocked from each other's restricted endpoints), and data ownership rules (an HR Officer cannot download a CV from a job created by a different officer; a Candidate cannot view another Candidate's application).

5.1.3 AI Pipeline Evaluation

The AI pipeline was evaluated on a test set of 300 CVs scored against a fixed job posting. Name extraction succeeded for all CVs; email extraction succeeded for 299 of 300 (one CV embedded its contact details inside an image, which PDFBox cannot extract as text). Skills extraction captured an average of 87% of the skills explicitly listed in each CV, with stronger recall for standard technical terms than for domain-specific jargon. Repeating the scoring call for the same CV against the same job posting produced scores differing by at most 5 points across all five dimensions, indicating that the LLM's structured output is stable under a fixed prompt and input.

An experienced HR Officer independently reviewed the same 300 CVs and produced their own shortlist decisions without seeing the AI output. The AI's recommendation label agreed with the HR Officer's decision in 80% cases. In the 20% discrepant cases, the AI rated 15% of candidates higher than the human reviewer (a candidate with strong technical skills but an unconventional career path) and 5% of candidates lower (a candidate with a well-formatted CV but limited depth in the specific required skills) - both consistent with known limitations of keyword- and structure-sensitive automated assessment.

Mean end-to-end processing time across the test set was 17.3 seconds per CV (18.4 s for PDF files, 16.1 s for DOCX files), comfortably within the 30-second limit.

5.1.4 Non-Functional Requirements Evaluation

NFR	Target	Measured Result	Status
API response time	< 2 s (50 concurrent users)	Mean 340 ms; peak 1.8 s	PASS
AI processing time per CV	< 30 s	Mean 17.3 s	PASS
Export generation	< 10 s	DOCX 4.2 s; PDF 6.7 s	PASS
Password hashing	BCrypt only	No plaintext passwords found in database	PASS
CV file access control	Ownership-verified only	No direct cloud URLs exposed in any response	PASS

Table 54: Non-Functional Requirements Evaluation Results

5.2 Discussion

The results indicate that an LLM-based pipeline can deliver consistent, reasonably accurate CV screening within practical time constraints, achieving 80% agreement with expert human judgement on a small but representative test set. This level of agreement is broadly consistent with prior published results for AI-assisted CV matching and classification, which typically report agreement or accuracy figures in the 80–94% range depending on task definition and CV format consistency.

The 20% disagreement cases observed during evaluation point to a known limitation of LLM-based scoring: candidates whose CVs do not explicitly state a skill or experience in conventional phrasing may be under-scored even when an experienced human reviewer would recognise the underlying competency from context. Equally, candidates whose CVs are well-organised but substantively weaker in the required skill area may be over-scored if the LLM is sensitive to formatting and clarity rather than purely to substantive content. This is consistent with documented findings in the fairness and bias literature on AI-based recruitment, which caution that even transparent, non-learned scoring algorithms can exhibit format-sensitivity unless carefully prompt-engineered and periodically audited against human judgement.

The system's reliance on a fixed weighting scheme (skills 35%, experience 30%, education 15%, certifications 10%, soft skills 10%) was a deliberate design decision to maximise transparency and auditability, in contrast to opaque semantic-similarity approaches used by some commercial systems. This trade-off proved appropriate for the target use case - small-to-medium HR teams without dedicated data science capacity - but does mean that the algorithm cannot automatically adapt its emphasis for roles where, for example, certifications should carry more weight than education level. This is identified as a direction for future work.

From an engineering perspective, the asynchronous, event-driven pipeline design proved effective at isolating failures: a malformed LLM response or transient API timeout for one CV in a batch did not block or delay the processing of the remaining CVs, satisfying the reliability requirement. The single most significant implementation challenge was ensuring reliable JSON output from the Gemini API despite explicit JSON-only prompting; this was mitigated through a response-sanitisation step that strips any non-JSON text surrounding the model's structured output before parsing.

Overall, the results support the conclusion that RecruitIQ successfully demonstrates the technical feasibility and practical value of AI-assisted CV screening for the target user segment, while also surfacing concrete, addressable limitations that inform the future work directions.

VI/ CONCLUSION & PERSPECTIVE

6.1 Summary of Contributions

This thesis has presented the design and implementation of RecruitIQ, a full-stack AI-powered CV screening and candidate ranking platform that addresses a real and documented problem in recruitment practice: the inability of HR teams to screen large volumes of CV applications consistently, objectively, and efficiently using manual methods.

The primary contributions of the work are:

- An end-to-end AI processing pipeline that integrates a large language model (Google Gemini) into a production-ready Spring Boot application, handling file ingestion, structured text extraction, multi-dimensional scoring, and recommendation generation as fully automated, asynchronous operations.
- A transparent, auditable scoring algorithm with fixed, human-readable weights across five dimensions (skills 35%, experience 30%, education 15%, certifications 10%, soft skills 10%), accompanied by per-dimension AI reasoning visible to HR Officers in the candidate detail view.
- A complete, deployable multi-role web application supporting HR Officers, Candidates, and Administrators through role-specific interfaces, with security enforced at both the API and data ownership levels.
- An empirical evaluation demonstrating that the system meets all defined non-functional requirements, achieves approximately 80% agreement with expert HR Officer judgement on recommendation labels, and processes CVs in under 20 seconds on average.

6.2 Limitations

Several limitations of the implemented system should be acknowledged:

- **LLM dependency:** The system's core functionality depends on the availability and responsiveness of the Google Gemini API. Degradation or unavailability of this external service directly impacts the AI processing pipeline. Mitigation via a fallback model is configured but does not address complete API outage scenarios.
- **Language bias in parsing:** The parsing prompts are in English and have been validated primarily on English-language CVs. Japanese-language CVs exhibit lower parsing recall, particularly for skills extraction, due to differences in standard CV structuring conventions. This is a known limitation for the target market.
- **Scoring subjectivity:** The fixed scoring weights reflect a generalised model of candidate evaluation. Different job types, industries, or organisational cultures may warrant different weight distributions. The system does not allow HR Officers to configure the weights per job posting.
- **No re-processing:** CVs that enter ERROR status cannot be re-processed. A manual retry mechanism would require careful handling of partial state (e.g., a Parsed Profile may exist while Score Record does not).
- **Single-server deployment:** The system architecture targets a single-server deployment. Horizontal scaling of the Spring Boot application would require externalisation of the async task executor to a message broker (e.g., RabbitMQ or Redis Streams), which is not implemented in this version.

6.3 Future Work

Based on the current implementation and identified limitations, the following directions are proposed for future development:

- **Interview scheduling module:** Allow HR Officers to assign interviewers, book calendar slots, and record interview outcomes within the RecruitIQ platform, eliminating the need for external coordination tools.
- **Job board integrations:** Publish job postings to Japanese job boards and LinkedIn directly from the RecruitIQ interface, and ingest applications submitted through those channels automatically.
- **Japanese CV parsing:** Develop a Japanese-specific parsing prompt or fine-tune a multilingual parsing model to improve extraction recall for Japanese-language CVs, which are predominant in the target market.
- **Configurable scoring weights:** Allow HR Officers to adjust the scoring weight distribution per job posting, enabling customisation for roles where, for example, certifications are more important than educational credentials.

Advanced analytics: Implement a pipeline analytics dashboard showing time-to-hire, screening funnel conversion rates, AI score distribution across positions, and HR decision outcomes - providing data-driven insights for continuous improvement of the hiring process.

6.4 Closing Remarks

RecruitIQ demonstrates that AI-powered CV screening is not only technically feasible but can be delivered as a complete, production-ready application by a small team within a university project timeline. The combination of a well-chosen LLM integration, a transparent scoring design, and a clean multi-role web application produces a tool that addresses a real operational problem faced by HR teams in Japan and similar markets. The system's design choices - event-driven async processing, fixed scoring weights with full AI reasoning transparency, and strict data access control - reflect lessons drawn from both the technical literature on AI recruitment systems and the practical constraints of the target user segment. The evaluation results confirm that the system meets its non-functional performance and security requirements and produces AI assessments that are broadly consistent with human expert judgement.

The work presented in this thesis contributes both a practical software artefact and an example of how large language model capabilities can be integrated responsibly into domain-specific applications - with transparency, auditability, and human oversight built into the design from the outset rather than added as an afterthought.

REFERENCES

1. Java SE 17 Edition: <https://docs.oracle.com/javase/17/>
2. Spring Boot 3.2: <https://spring.io/projects/spring-boot>
3. Spring Security 6: <https://spring.io/projects/spring-security>
4. Spring Data JPA: <https://spring.io/projects/spring-data-jpa>
5. PostgreSQL Database Driver: <https://www.postgresql.org/>
6. Cloudinary HTTP Client (Cloudinary 1.36.0): <https://cloudinary.com/>
7. Apache PDFBox 3.0.2: <https://www.apache.org/>
8. Apache POI 5.2.5: <https://poi.apache.org/>
9. Springdoc OpenAPI UI 2.3.0: <https://springdoc.org/>
10. Vite : <https://vite.dev/>
11. React : <https://react.dev/>